DEVI AHILYA VISHWAVIDYALAYA (DAVV), INDORE

School of Computer Science & Information Technology

Department of Computer Science & Information Technology

Programme: M.Tech (Information Architecture & Software Engineering)

LABORATORY MANUAL

Internet Programming Using Java

Course Code: CS-5225

Semester 1, Year 1

Submitted By:

Name: ROHIT BHURE

Roll No.: 2610611

Programme: M.Tech (IA&SE)

Semester: 1 (Year 1)

Submitted To:

Name: Mr. Chetan Awasthi

Designation: Professor

Department: School of Computer Science & Information Technology

_____

Signature

Academic Session: 2026–2027

CERTIFICATE

This is to certify that ROHIT BHURE, Roll No. 2610611, has satisfactorily completed all the experiments of Internet Programming Using Java (Course Code: CS-5225) as prescribed by Devi Ahilya Vishwavidyalaya (DAVV) for M.Tech (Information Architecture & Software Engineering), Semester 1, during the academic session 2026–2027.

Faculty / Lab Incharge
Name: Mr. Chetan Awasthi
Date: _____

INDEX

Exp. No.	Title of Experiment	Page No.	Date	Signature
1	Create a program that reads jagged array dimensions from command-line arguments, dynamically creates the array, and demonstrates implicit type promotion during element assignment.	5		
2	Write an overloaded process() method where one version accepts a multi-dimensional array and another accepts varargs. Demonstrate recursion with explicit narrowing type casting and trace the call stack.	7		
3	Design a class to demonstrate variable shadowing using a global String and a local StringBuffer, displaying their hash codes in each loop iteration.	9		
4	Implement a constant-time string comparison using charAt() without using the equals() method.	10		
5	Write a program that creates a deep copy of a 3D array, rotates (or flips) the copy, and returns it without modifying the original array.	11		
6	Create overloaded methods calculate(int... nums) and calculate(double num, int... nums), then invoke them using short, char, and float values to demonstrate method resolution.	13		
7	Write a program to manually sort an array of strings by converting them into byte arrays without using built-in sorting methods.	15		
8	Create a recursive program that tracks method calls, terminates recursion based on a condition, and continues execution iteratively to avoid StackOverflowError.	17		
9	Design a program that demonstrates upcasting and downcasting using an array of objects and updates object states using an enhanced for loop.	18		
10	Write a program that performs repeated string concatenation inside a loop to demonstrate heap allocation and garbage collection through reference reassignment.	20		
11	Develop a University Management System using multilevel inheritance, abstract classes, method overriding, and runtime polymorphism.	22		
12	Create a multi-package Banking System demonstrating custom packages, access modifiers, package importing, and CLASSPATH configuration.	24		
13	Design a custom utility package for matrix and statistical operations and execute it using CLASSPATH.	26		
14	Implement an Online Examination System using user-defined exceptions, exception propagation, and nested exception handling.	28		
15	Develop a multithreaded Railway Reservation System demonstrating thread priorities, synchronization, and thread life cycle.	30		
16	Implement the Producer–Consumer problem using multiple threads with wait(), notify(), and synchronization.	32		
17	Create a Java Applet to draw geometric shapes using mouse events and demonstrate the complete applet life cycle.	34		
18	Develop a JDBC-based Student Management System implementing CRUD operations, transactions, and PreparedStatement.	36		
19	Create a multithreaded Banking System that performs synchronized database transactions using JDBC.	38		
20	Develop a Library Management System integrating inheritance, packages, exception handling, multithreading, interfaces, and JDBC.	40		
21	Develop a login system using Java Servlets that authenticates users, manages sessions, and restricts access to protected resources using deployment descriptors.	42		
22	Create a multi-module web application with multiple servlets configured entirely through web.xml, demonstrating servlet mapping, initialization parameters, and load-on-startup.	44		
23	Design an online examination system using Servlets that processes dynamic forms, validates user input, maintains session state, and generates a result report.	46		
24	Develop a shopping cart application using Servlets that demonstrates all session tracking techniques (Cookies, URL Rewriting, Hidden Fields, and HttpSession) and compares their behaviour.	48		
25	Implement a file upload and download system using Servlets that validates uploaded files, stores metadata, and handles exceptions gracefully.	50		
26	Create a servlet-based Employee Management System implementing CRUD operations with HTML forms, request dispatching, and configurable servlet initialization parameters.	52		

Exp. No.	Title of Experiment	Page No.	Date	Signature
27	Develop a secure online banking portal using GenericServlet and HttpServlet to demonstrate authentication, authorization, session timeout, and servlet life cycle methods.	54		
28	Build a visitor analytics application that records client requests, browser information, session duration, and page visits using Servlet APIs and deployment descriptor configuration.	56		
29	Create a servlet application that dynamically generates reports based on user input, uses request forwarding and redirection, and manages concurrent client requests efficiently.	58		
30	Develop a complete e-commerce web application using Servlets, deployment descriptors, session management, and servlet life cycle methods, with configurable initialization parameters defined in web.xml.	60		
31	Develop a JSP-based Student Management System that performs CRUD operations using JavaBeans, JDBC, and MySQL while separating business logic from presentation logic.	62		
32	Create a secure JSP login application that authenticates users using a database, manages sessions, and handles invalid login attempts with custom error pages.	64		
33	Design an online banking portal using JSP, JavaBeans, and JDBC to perform account management, fund transfers, and transaction history with proper exception handling.	66		
34	Develop a JSP application demonstrating the complete JSP lifecycle, directives, scripting elements, standard actions, and implicit objects in a single project.	68		
35	Create a database-driven Inventory Management System using JSP and JDBC that supports Statement, PreparedStatement, and CallableStatement, and compares their usage.	70		
36	Implement a JSP-based Employee Management System that connects to Oracle, MySQL, or MS SQL Server and performs CRUD operations using JavaBeans and MVC architecture.	72		
37	Develop a JSP application that uses session management to implement a multi-step online examination system with automatic session timeout and recovery.	75		
38	Create a dynamic reporting system using JSP and JDBC that executes parameterized database queries and displays formatted reports with robust exception handling.	77		
39	Design a JSP-based Library Management System using JavaBeans, JDBC, and MVC architecture, supporting book issue/return operations and database transaction management.	79		
40	Develop a complete JSP-based E-Commerce application integrating JavaBeans, session management, JDBC, MVC architecture, database CRUD operations, and comprehensive error and exception handling.	82		
41	Develop a JSP-based Student Management System that performs CRUD operations using JavaBeans, JDBC, and MySQL while separating business logic from presentation logic.	85		
42	Create a secure JSP login application that authenticates users using a database, manages sessions, and handles invalid login attempts with custom error pages.	87		
43	Design an online banking portal using JSP, JavaBeans, and JDBC to perform account management, fund transfers, and transaction history with proper exception handling.	89		
44	Develop a JSP application demonstrating the complete JSP lifecycle, directives, scripting elements, standard actions, and implicit objects in a single project.	91		
45	Create a database-driven Inventory Management System using JSP and JDBC that supports Statement, PreparedStatement, and CallableStatement, and compares their usage.	93		
46	Implement a JSP-based Employee Management System that connects to Oracle, MySQL, or MS SQL Server and performs CRUD operations using JavaBeans and MVC architecture.	95		
47	Develop a JSP application that uses session management to implement a multi-step online examination system with automatic session timeout and recovery.	98		
48	Create a dynamic reporting system using JSP and JDBC that executes parameterized database queries and displays formatted reports with robust exception handling.	100		

Exp. No.	Title of Experiment	Page No.	Date	Signature
49	Design a JSP-based Library Management System using JavaBeans, JDBC, and MVC architecture, supporting book issue/return operations and database transaction management.	102		
50	Develop a complete JSP-based E-Commerce application integrating JavaBeans, session management, JDBC, MVC architecture, database CRUD operations, and comprehensive error and exception handling.	105		
51	Write a Java program to demonstrate the concept of inheritance using a parent class and a child class. Create methods in both classes and access the parent class properties and methods through the child class.	108		
52	Write a Java program to perform basic arithmetic operations (Addition, Subtraction, Multiplication, Division and Modulus) on two numbers using user input.	110		
53	Write a Java program to create a user-defined package, define a class inside the package, and access its methods from another Java class using the import statement.	112		
54	Write a Java program to establish a database connection with MySQL using JDBC by loading the MySQL Connector/J driver. Perform the following CRUD operations on a Student table: Create – Insert a new student record; Read – Display all student records; Update – Update an existing student record; Delete – Delete a student record.	114		
55	Write a Java Servlet program using Apache Tomcat to accept user input through an HTML form and display the submitted data on a web page. Demonstrate the use of doGet() and doPost() methods.	116		

Experiment No. 1

Create a program that reads jagged array dimensions from command-line arguments, dynamically creates the array, and demonstrates implicit type promotion during element assignment.

Aim:

To create a jagged array whose dimensions are supplied through command-line arguments and to demonstrate implicit (widening) type promotion while assigning elements.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A jagged array is an array of arrays in which every row can have a different length. In Java the outer array is created first and each row is allocated separately with new. Command-line arguments reach main() as String[] args and must be converted with Integer.parseInt(). Implicit type promotion (widening conversion) is performed automatically by the compiler when a smaller type such as byte, short or char is assigned to a wider type such as int, long, float or double, with no loss of magnitude.

Procedure / Algorithm:

1. Read the number of rows from args[0] and the length of each row from args[1..n].
2. Create the outer array int[rows][] and allocate each row with its own length.
3. Assign byte, short and char values to int elements to show widening of the values.
4. Assign an int element to double and float variables to show further promotion.
5. Print every row of the jagged array and the promoted values.

Code / Program:

```
public class JaggedArray {
    public static void main(String[] args) {
        int rows = Integer.parseInt(args[0]);
        int[][] arr = new int[rows][];
        for (int i = 0; i < rows; i++) {
            int cols = Integer.parseInt(args[i + 1]);
            arr[i] = new int[cols];
        }
        byte b = 5;
        short s = 300;
        char c = 'A';
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                if (j % 3 == 0) arr[i][j] = b;       // byte -> int
                else if (j % 3 == 1) arr[i][j] = s;  // short -> int
                else arr[i][j] = c;                  // char -> int
            }
        }
        for (int i = 0; i < arr.length; i++) {
            System.out.print("Row " + i + " (" + arr[i].length + "): ");
            for (int v : arr[i]) System.out.print(v + " ");
            System.out.println();
        }
        double d = arr[0][0];  // int -> double
        float f = c;           // char -> float
        long l = arr[0][1];    // int -> long
        System.out.println("double=" + d + ", float=" + f + ", long=" + l);
    }
}
```

Output:

```
Row 0 (2): 5 300
Row 1 (4): 5 300 65 5
Row 2 (3): 5 300 65
```

```
double=5.0, float=65.0, long=300
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on jagged arrays and implicit type promotion has been implemented and verified.

Conclusion:

Thus, jagged arrays and implicit type promotion was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 2

Write an overloaded process() method where one version accepts a multi-dimensional array and another accepts varargs. Demonstrate recursion with explicit narrowing type casting and trace the call stack.

Aim:

To implement overloaded process() methods for a 2D array and varargs, and to demonstrate recursion with explicit narrowing casts while tracing the call stack.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Method overloading lets a class contain several methods with the same name but different parameter lists. The compiler selects a method at compile time by matching argument types; a varargs parameter (int...) is treated as an array and is considered only after fixed-arity methods fail to match. Recursion is a method calling itself, and every call is pushed on the JVM call stack until the base case is reached. Explicit narrowing conversion, written as (byte) or (int), converts a wider type to a narrower type and may lose information.

Procedure / Algorithm:

1. Define process(int[][] m) that adds all elements of a 2D array.
2. Define process(int... v) that reports the number of arguments received.
3. Write a recursive factorial() that prints entry and exit with indentation to trace the stack.
4. Narrow the long result to byte and a double to int using explicit casts.
5. Call all methods from main() and observe the output.

Code / Program:

```
public class ProcessDemo {
    static int depth = 0;

    static void process(int[][] m) {
        int s = 0;
        for (int[] r : m) for (int x : r) s += x;
        System.out.println("process(int[][]) sum = " + s);
    }

    static void process(int... v) {
        System.out.println("process(int...) count = " + v.length);
    }

    static long factorial(int n) {
        depth++;
        String pad = " ".repeat(depth * 2);
        System.out.println(pad + "-> factorial(" + n + ") depth=" + depth);
        long r = (n <= 1) ? 1 : n * factorial(n - 1);
        System.out.println(pad + "<- return " + r);
        depth--;
        return r;
    }

    public static void main(String[] args) {
        process(new int[][]{{1, 2}, {3, 4, 5}});
        process(1, 2, 3);
        process();
        long f = factorial(5);
        byte b = (byte) f;        // narrowing: 120 fits
        byte big = (byte) 300;    // narrowing: overflow
        int i = (int) 3.99;       // fraction truncated
        System.out.println("f=" + f + ", (byte)f=" + b
            + ", (byte)300=" + big + ", (int)3.99=" + i);
```

```
    }
}
```

Output:

```
process(int[][]) sum = 15
process(int...) count = 3
process(int...) count = 0
  -> factorial(5) depth=1
    -> factorial(4) depth=2
      -> factorial(3) depth=3
        -> factorial(2) depth=4
          -> factorial(1) depth=5
          <- return 1
        <- return 2
      <- return 6
    <- return 24
  <- return 120
f=120, (byte)f=120, (byte)300=44, (int)3.99=3
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on overloading, recursion and narrowing casts has been implemented and verified.

Conclusion:

Thus, overloading, recursion and narrowing casts was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 3

Design a class to demonstrate variable shadowing using a global String and a local StringBuffer, displaying their hash codes in each loop iteration.

Aim:

To demonstrate variable shadowing by declaring a local StringBuffer with the same name as a global String and to display the hash codes in every loop iteration.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Variable shadowing occurs when a local variable has the same name as a variable of an enclosing scope, so the local one hides the outer one inside its block. The hidden static field can still be reached with the class name qualifier. String is immutable, so its hashCode() depends on the contents, whereas StringBuffer does not override hashCode(); it returns an identity-based value that stays constant while the same object is modified by append().

Procedure / Algorithm:

1. Declare a static String named data as the global variable.
2. Inside main(), declare a local StringBuffer also named data.
3. In a loop, append the loop counter to the local buffer.
4. Print the local and global values with their hashCode() in each iteration.
5. Access the global variable using ShadowDemo.data and compare the outputs.

Code / Program:

```
public class ShadowDemo {
    static String data = "Global";

    public static void main(String[] args) {
        System.out.println("Before shadowing: data=" + data
            + ", hash=" + data.hashCode());
        StringBuffer data = new StringBuffer("Local");   // shadows global
        for (int i = 1; i <= 3; i++) {
            data.append(i);
            System.out.println("Iteration " + i
                + " | local=" + data + ", hash=" + data.hashCode()
                + " | global=" + ShadowDemo.data
                + ", hash=" + ShadowDemo.data.hashCode());
        }
    }
}
```

Output:

```
Before shadowing: data=Global, hash=2135814083
Iteration 1 | local=Local1, hash=1625635731 | global=Global, hash=2135814083
Iteration 2 | local=Local12, hash=1625635731 | global=Global, hash=2135814083
Iteration 3 | local=Local123, hash=1625635731 | global=Global, hash=2135814083
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on variable shadowing, String immutability and hash codes has been implemented and verified.

Conclusion:

Thus, variable shadowing, String immutability and hash codes was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 4

Implement a constant-time string comparison using charAt() without using the equals() method.

Aim:

To implement a constant-time string comparison method using charAt() without calling equals().

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A normal comparison stops at the first mismatching character, so its running time leaks information about how many leading characters match; this is exploited in timing attacks on passwords and tokens. A constant-time comparison always examines every character of the common length and accumulates differences with the bitwise XOR and OR operators, returning the result only at the end. The method charAt(i) returns the character at index i and is used to compare the strings character by character.

Procedure / Algorithm:

1. Initialise diff with a.length() XOR b.length() so that unequal lengths are detected.
2. Loop over the common length without any early exit.
3. For every index compute a.charAt(i) ^ b.charAt(i) and OR it into diff.
4. Return true only if diff is zero.
5. Test with equal, different and unequal-length strings.

Code / Program:

```
public class SafeCompare {
    static boolean compare(String a, String b) {
        int diff = a.length() ^ b.length();
        int n = Math.min(a.length(), b.length());
        for (int i = 0; i < n; i++) {
            diff |= a.charAt(i) ^ b.charAt(i);
        }
        return diff == 0;
    }

    public static void main(String[] args) {
        System.out.println(compare("secret123", "secret123"));
        System.out.println(compare("secret123", "secret124"));
        System.out.println(compare("secret", "secret123"));
        System.out.println(compare("", ""));
    }
}
```

Output:

```
true
false
false
true
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on constant-time comparison using charAt() has been implemented and verified.

Conclusion:

Thus, constant-time comparison using charAt() was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 5

Write a program that creates a deep copy of a 3D array, rotates (or flips) the copy, and returns it without modifying the original array.

Aim:

To create a deep copy of a 3D array, rotate every layer of the copy by 90 degrees and return it while the original array stays unchanged.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Assigning an array reference or calling clone() on a multi-dimensional array produces only a shallow copy, so inner arrays are shared with the original. A deep copy allocates every level separately and copies the primitive values. A 90-degree clockwise rotation of an r x c matrix maps the element at (i, j) to (j, r-1-i) of a c x r matrix. Working only on the copy guarantees that the original data is not modified.

Procedure / Algorithm:

1. Write deepCopy() that allocates new arrays at all three levels and copies each row with clone().
2. Write rotate() that first calls deepCopy().
3. For every layer create a new c x r matrix and store element (i, j) at (j, r-1-i).
4. Return the rotated copy.
5. Print the original and the rotated array with Arrays.deepToString().

Code / Program:

```
import java.util.Arrays;

public class Rotate3D {
    static int[][][] deepCopy(int[][][] s) {
        int[][][] c = new int[s.length][][];
        for (int i = 0; i < s.length; i++) {
            c[i] = new int[s[i].length][];
            for (int j = 0; j < s[i].length; j++)
                c[i][j] = s[i][j].clone();
        }
        return c;
    }

    static int[][][] rotate(int[][][] src) {
        int[][][] c = deepCopy(src);
        for (int k = 0; k < c.length; k++) {
            int r = c[k].length, w = c[k][0].length;
            int[][] t = new int[w][r];
            for (int i = 0; i < r; i++)
                for (int j = 0; j < w; j++)
                    t[j][r - 1 - i] = c[k][i][j];
            c[k] = t;
        }
        return c;
    }

    public static void main(String[] args) {
        int[][][] a = {{{1, 2, 3}, {4, 5, 6}}, {{7, 8, 9}, {10, 11, 12}}};
        int[][][] r = rotate(a);
        System.out.println("Original: " + Arrays.deepToString(a));
        System.out.println("Rotated : " + Arrays.deepToString(r));
    }
}
```

Output:

```
Original: [[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]]
```

```
Rotated : [[[4, 1], [5, 2], [6, 3]], [[10, 7], [11, 8], [12, 9]]]
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on deep copying and rotating multi-dimensional arrays has been implemented and verified.

Conclusion:

Thus, deep copying and rotating multi-dimensional arrays was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 6

Create overloaded methods calculate(int... nums) and calculate(double num, int... nums), then invoke them using short, char, and float values to demonstrate method resolution.

Aim:

To create overloaded varargs methods and to observe how the compiler resolves calls made with short, char and float arguments.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

When a method call is compiled, Java resolves the overload in phases: exact match and widening without boxing or varargs first, then boxing, and finally variable-arity methods. For short and char arguments the widening primitive conversion to int is possible, so calculate(int...) is applicable. A float argument cannot be widened to int but can be widened to double, so calculate(double, int...) is selected. When more than one varargs method is applicable, the most specific one is chosen.

Procedure / Algorithm:

1. Define calculate(int... nums) that returns the sum of its arguments.
2. Define calculate(double num, int... nums) that adds num to the sum.
3. Declare short, char and float variables.
4. Call both methods with single and multiple arguments of these types.
5. Note which version is invoked for each call.

Code / Program:

```
public class OverloadDemo {
    static void calculate(int... nums) {
        int s = 0;
        for (int n : nums) s += n;
        System.out.println("calculate(int...) -> sum = " + s);
    }

    static void calculate(double num, int... nums) {
        double s = num;
        for (int n : nums) s += n;
        System.out.println("calculate(double, int...) -> sum = " + s);
    }

    public static void main(String[] args) {
        short sh = 10;
        char ch = 'A';
        float fl = 2.5f;
        calculate(sh);
        calculate(ch);
        calculate(fl);
        calculate(sh, ch);
        calculate(fl, sh, ch);
    }
}
```

Output:

```
calculate(int...) -> sum = 10
calculate(int...) -> sum = 65
calculate(double, int...) -> sum = 2.5
calculate(int...) -> sum = 75
calculate(double, int...) -> sum = 77.5
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on overload resolution with varargs and widening has been implemented and verified.

Conclusion:

Thus, overload resolution with varargs and widening was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 7

Write a program to manually sort an array of strings by converting them into byte arrays without using built-in sorting methods.

Aim:

To sort an array of strings manually by converting each string into a byte array and comparing the bytes, without using built-in sort methods.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A String can be converted to its byte representation with getBytes(). Comparing two strings lexicographically is equivalent to comparing their byte arrays element by element, treating each byte as an unsigned value (b & 0xFF); if all common bytes are equal the shorter array is smaller. Insertion sort builds a sorted portion by inserting each element at its correct position and needs no library support.

Procedure / Algorithm:

1. Convert every string of the input array into a byte[] using getBytes().
2. Write cmp() that compares two byte arrays lexicographically.
3. Apply insertion sort on the array of byte arrays using cmp().
4. Convert each byte array back with new String(bytes).
5. Display the strings before and after sorting.

Code / Program:

```
public class ByteSort {
    static int cmp(byte[] a, byte[] b) {
        int n = Math.min(a.length, b.length);
        for (int i = 0; i < n; i++)
            if (a[i] != b[i]) return (a[i] & 0xFF) - (b[i] & 0xFF);
        return a.length - b.length;
    }

    public static void main(String[] args) {
        String[] s = {"banana", "apple", "cherry", "mango", "apricot"};
        byte[][] bs = new byte[s.length][];
        for (int i = 0; i < s.length; i++) bs[i] = s[i].getBytes();

        for (int i = 1; i < bs.length; i++) {
            byte[] key = bs[i];
            int j = i - 1;
            while (j >= 0 && cmp(bs[j], key) > 0) {
                bs[j + 1] = bs[j];
                j--;
            }
            bs[j + 1] = key;
        }
        System.out.print("Sorted: ");
        for (byte[] b : bs) System.out.print(new String(b) + " ");
        System.out.println();
    }
}
```

Output:

```
Sorted: apple apricot banana cherry mango
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on manual sorting using byte arrays has been implemented and verified.

Conclusion:

Thus, manual sorting using byte arrays was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 8

Create a recursive program that tracks method calls, terminates recursion based on a condition, and continues execution iteratively to avoid StackOverflowError.

Aim:

To write a recursive program that counts its calls, stops recursing when a limit is reached, and completes the remaining work iteratively to avoid StackOverflowError.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Each recursive call occupies a stack frame, and very deep recursion throws StackOverflowError because the JVM stack is limited. A hybrid approach counts the calls and, when a safe depth is reached, switches from recursion to a loop that finishes the remaining computation. The result is identical to pure recursion but uses a bounded amount of stack space. A static counter is used to track the number of method invocations.

Procedure / Algorithm:

1. Declare a static counter calls and a constant LIMIT.
2. In sum(n, acc) increment calls and return acc when n is 0.
3. If calls reaches LIMIT, finish the remaining additions with a for loop.
4. Otherwise call sum(n - 1, acc + n) recursively.
5. Run with a large n and print the result and number of recursive calls.

Code / Program:

```
public class HybridRecursion {
    static int calls = 0;
    static final int LIMIT = 1000;

    static long sum(int n, long acc) {
        calls++;
        if (n == 0) return acc;
        if (calls >= LIMIT) {            // stop recursion, continue iteratively
            long r = acc;
            for (int i = n; i > 0; i--) r += i;
            return r;
        }
        return sum(n - 1, acc + n);
    }

    public static void main(String[] args) {
        int n = 100000;
        long result = sum(n, 0);
        System.out.println("Sum of 1.." + n + " = " + result);
        System.out.println("Recursive calls made = " + calls);
    }
}
```

Output:

```
Sum of 1..100000 = 5000050000
Recursive calls made = 1000
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on controlling recursion depth by switching to iteration has been implemented and verified.

Conclusion:

Thus, controlling recursion depth by switching to iteration was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 9

Design a program that demonstrates upcasting and downcasting using an array of objects and updates object states using an enhanced for loop.

Aim:

To demonstrate upcasting and downcasting with an array of objects and to update their states using an enhanced for loop.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Upcasting assigns a subclass object to a superclass reference and is done implicitly; only members declared in the superclass are accessible through that reference. Downcasting converts the reference back to the subclass type with an explicit cast so that subclass members can be used, and it is safe only after checking with instanceof, otherwise ClassCastException is thrown. The enhanced for loop iterates over every element of an array without an index.

Procedure / Algorithm:

1. Create class Animal and subclasses Dog and Cat with their own methods.
2. Store Dog, Cat and Animal objects in an Animal[] array (upcasting).
3. Use an enhanced for loop to update the energy field of every object.
4. Check each object with instanceof and downcast it to call fetch() or climb().
5. Show a failed downcast being caught as ClassCastException.

Code / Program:

```
class Animal {
    String name; int energy = 10;
    Animal(String n) { name = n; }
    void speak() { System.out.println(name + " makes a sound"); }
}
class Dog extends Animal {
    Dog(String n) { super(n); }
    void speak() { System.out.println(name + " barks"); }
    void fetch() { System.out.println(name + " fetches the ball"); }
}
class Cat extends Animal {
    Cat(String n) { super(n); }
    void speak() { System.out.println(name + " meows"); }
    void climb() { System.out.println(name + " climbs the tree"); }
}
public class CastDemo {
    public static void main(String[] args) {
        Animal[] zoo = {new Dog("Rex"), new Cat("Tom"), new Animal("Generic")};
        for (Animal a : zoo) {
            a.energy += 5;                          // state update
            a.speak();
            if (a instanceof Dog) ((Dog) a).fetch();   // downcast
            else if (a instanceof Cat) ((Cat) a).climb();
            System.out.println("  energy = " + a.energy);
        }
        try {
            Dog d = (Dog) zoo[1];                   // invalid downcast
        } catch (ClassCastException e) {
            System.out.println("ClassCastException caught");
        }
    }
}
```

Output:

```
Rex barks
Rex fetches the ball
```

```
  energy = 15
Tom meows
Tom climbs the tree
  energy = 15
Generic makes a sound
  energy = 15
ClassCastException caught
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on upcasting, downcasting and the enhanced for loop has been implemented and verified.

Conclusion:

Thus, upcasting, downcasting and the enhanced for loop was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 10

Write a program that performs repeated string concatenation inside a loop to demonstrate heap allocation and garbage collection through reference reassignment.

Aim:

To show heap allocation caused by repeated string concatenation in a loop and garbage collection of unreachable strings after reference reassignment.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Strings are immutable, so every concatenation with + creates a new String object on the heap and the variable is reassigned to it; the old object becomes unreachable. Such objects are eligible for garbage collection, which reclaims their memory. System.identityHashCode() shows that each iteration produces a different object. A WeakReference does not keep its referent alive, so it can be used to observe whether the collector has reclaimed the object after System.gc().

Procedure / Algorithm:

1. Create a String with new String("start") and keep a WeakReference to it.
2. Concatenate inside a loop and print the identity hash code of s each time.
3. Observe that the reference points to a new object in every iteration.
4. Call System.gc() and check whether the weak reference has been cleared.
5. Print the memory used with Runtime.

Code / Program:

```
import java.lang.ref.WeakReference;

public class ConcatGC {
    public static void main(String[] args) throws Exception {
        String s = new String("start");
        WeakReference<String> weak = new WeakReference<>(s);
        for (int i = 1; i <= 5; i++) {
            s = s + i;                          // new object each time
            System.out.println("i=" + i + ", s=" + s
                + ", id=" + System.identityHashCode(s));
        }
        System.gc();
        Thread.sleep(200);
        System.out.println("Original string collected: " + (weak.get() == null));
        Runtime rt = Runtime.getRuntime();
        System.out.println("Used memory (KB): "
            + (rt.totalMemory() - rt.freeMemory()) / 1024);
    }
}
```

Output:

```
i=1, s=start1, id=1625635731
i=2, s=start12, id=1414644648
i=3, s=start123, id=640070680
i=4, s=start1234, id=1510467688
i=5, s=start12345, id=1995265320
Original string collected: true
Used memory (KB): 1399
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on heap allocation and garbage collection of unreachable strings has been implemented and verified.

Conclusion:

Thus, heap allocation and garbage collection of unreachable strings was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 11

Develop a University Management System using multilevel inheritance, abstract classes, method overriding, and runtime polymorphism.

Aim:

To develop a University Management System that uses multilevel inheritance, abstract classes, method overriding and runtime polymorphism.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Inheritance allows a class to acquire the members of another class; in multilevel inheritance a class is derived from a derived class (Person -> Staff -> Professor). An abstract class cannot be instantiated and may declare abstract methods that subclasses must implement. Method overriding redefines a superclass method in a subclass, and runtime polymorphism means that the version executed is decided by the actual object type at run time through a superclass reference (dynamic method dispatch).

Procedure / Algorithm:

1. Create abstract class Person with name and abstract method role().
2. Derive Staff from Person and Professor from Staff (multilevel inheritance).
3. Derive Student from Person and override role() and details() in every class.
4. Store objects in a Person[] array and call the methods through the Person reference.
5. Observe the overridden methods executed at run time.

Code / Program:

```
abstract class Person {
    protected String name;
    Person(String name) { this.name = name; }
    abstract String role();
    void details() { System.out.println(role() + ": " + name); }
}
class Staff extends Person {
    protected double salary;
    Staff(String n, double s) { super(n); salary = s; }
    String role() { return "Staff"; }
    void details() { super.details(); System.out.println("  Salary: " + salary); }
}
class Professor extends Staff {
    private String dept;
    Professor(String n, double s, String d) { super(n, s); dept = d; }
    String role() { return "Professor"; }
    void details() { super.details(); System.out.println("  Department: " + dept); }
}
class Student extends Person {
    private int roll;
    Student(String n, int r) { super(n); roll = r; }
    String role() { return "Student"; }
    void details() { super.details(); System.out.println("  Roll No: " + roll); }
}
public class University {
    public static void main(String[] args) {
        Person[] p = {
            new Professor("Dr. Mehta", 95000, "Computer Science"),
            new Staff("Anita Rao", 32000),
            new Student("Rohit", 2610611)
        };
        for (Person x : p) x.details();   // runtime polymorphism
    }
}
```

Output:

```
Professor: Dr. Mehta
  Salary: 95000.0
  Department: Computer Science
Staff: Anita Rao
  Salary: 32000.0
Student: Rohit
  Roll No: 2610611
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on multilevel inheritance, abstract classes and runtime polymorphism has been implemented and verified.

Conclusion:

Thus, multilevel inheritance, abstract classes and runtime polymorphism was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 12

Create a multi-package Banking System demonstrating custom packages, access modifiers, package importing, and CLASSPATH configuration.

Aim:

To create a multi-package Banking System that demonstrates custom packages, access modifiers, package importing and CLASSPATH configuration.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A package groups related classes and avoids name conflicts; it maps to a directory structure. Access modifiers control visibility: private (same class), default (same package), protected (package and subclasses) and public (everywhere). Classes of another package are used through the import statement. The CLASSPATH environment variable, or the -cp option, tells the compiler and the JVM where to find user-defined packages.

Procedure / Algorithm:

1. Create folder bank with sub-packages bank/model and bank/service.
2. Write Account (public class with private, protected and default members) in bank.model.
3. Write BankService in bank.service that imports bank.model.Account.
4. Write the Main class in package bank.app and import both packages.
5. Compile with javac -d out and run with java -cp out bank.app.Main (or set CLASSPATH).

Code / Program:

```
// FILE: bank/model/Account.java
package bank.model;
public class Account {
    private double balance;          // private: only this class
    protected String holder;         // protected: package + subclasses
    String branch = "Indore";        // default: same package only
    public final int accNo;          // public: everywhere
    public Account(int accNo, String holder, double bal) {
        this.accNo = accNo; this.holder = holder; this.balance = bal;
    }
    public double getBalance() { return balance; }
    public void deposit(double a) { if (a > 0) balance += a; }
    public boolean withdraw(double a) {
        if (a > 0 && a <= balance) { balance -= a; return true; }
        return false;
    }
    public String branch() { return branch; }
}
// FILE: bank/service/BankService.java
package bank.service;
import bank.model.Account;
public class BankService {
    public void transfer(Account from, Account to, double amt) {
        if (from.withdraw(amt)) { to.deposit(amt); System.out.println("Transferred " +
amt); }
        else System.out.println("Insufficient balance");
    }
}
// FILE: bank/app/Main.java
package bank.app;
import bank.model.Account;
import bank.service.BankService;
public class Main {
    public static void main(String[] args) {
        Account a = new Account(101, "Rohit", 5000);
        Account b = new Account(102, "Amit", 2000);
```

```
        new BankService().transfer(a, b, 1500);
        new BankService().transfer(a, b, 9000);
        System.out.println(a.accNo + " balance = " + a.getBalance());
        System.out.println(b.accNo + " balance = " + b.getBalance());
        System.out.println("Branch = " + a.branch());
    }
}
```

Output:

```
Transferred 1500.0
Insufficient balance
101 balance = 3500.0
102 balance = 3500.0
Branch = Indore
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on packages, access modifiers and CLASSPATH has been implemented and verified.

Conclusion:

Thus, packages, access modifiers and CLASSPATH was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 13

Design a custom utility package for matrix and statistical operations and execute it using CLASSPATH.

Aim:

To design a custom utility package containing matrix and statistical operations and to execute it using CLASSPATH.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A utility package bundles reusable static methods that can be imported by any program. Matrix operations such as addition, multiplication and transpose work on two-dimensional arrays, while statistical operations such as mean, median and standard deviation work on one-dimensional data. After compiling with javac -d, the package directory can be placed anywhere and made visible to the JVM by setting CLASSPATH (set CLASSPATH=C:\lib;. on Windows or export CLASSPATH=/home/lib:. on Linux).

Procedure / Algorithm:

1. Create package util with classes MatrixUtil and StatUtil containing static methods.
2. Compile them with javac -d C:\lib util/*.java.
3. Set CLASSPATH to include C:\lib and the current directory.
4. Write a test class that imports util.* and calls the methods.
5. Compile and run the test class and verify results.

Code / Program:

```
// FILE: util/MatrixUtil.java
package util;
public class MatrixUtil {
    public static int[][] add(int[][] a, int[][] b) {
        int[][] c = new int[a.length][a[0].length];
        for (int i = 0; i < a.length; i++)
            for (int j = 0; j < a[0].length; j++) c[i][j] = a[i][j] + b[i][j];
        return c;
    }
    public static int[][] multiply(int[][] a, int[][] b) {
        int[][] c = new int[a.length][b[0].length];
        for (int i = 0; i < a.length; i++)
            for (int j = 0; j < b[0].length; j++)
                for (int k = 0; k < b.length; k++) c[i][j] += a[i][k] * b[k][j];
        return c;
    }
    public static int[][] transpose(int[][] a) {
        int[][] t = new int[a[0].length][a.length];
        for (int i = 0; i < a.length; i++)
            for (int j = 0; j < a[0].length; j++) t[j][i] = a[i][j];
        return t;
    }
}
// FILE: util/StatUtil.java
package util;
import java.util.Arrays;
public class StatUtil {
    public static double mean(double[] d) {
        double s = 0; for (double x : d) s += x; return s / d.length;
    }
    public static double median(double[] d) {
        double[] c = d.clone(); Arrays.sort(c); int n = c.length;
        return n % 2 == 1 ? c[n / 2] : (c[n / 2 - 1] + c[n / 2]) / 2;
    }
    public static double stdDev(double[] d) {
        double m = mean(d), s = 0;
```

```
        for (double x : d) s += (x - m) * (x - m);
        return Math.sqrt(s / d.length);
    }
}
// FILE: UtilTest.java
import util.*;
import java.util.Arrays;
public class UtilTest {
    public static void main(String[] args) {
        int[][] a = {{1, 2}, {3, 4}}, b = {{5, 6}, {7, 8}};
        System.out.println("Sum      : " + Arrays.deepToString(MatrixUtil.add(a, b)));
        System.out.println("Product  : " + Arrays.deepToString(MatrixUtil.multiply(a,
b)));
        System.out.println("Transpose: " +
Arrays.deepToString(MatrixUtil.transpose(a)));
        double[] d = {4, 8, 6, 5, 3, 7};
        System.out.println("Mean=" + StatUtil.mean(d) + ", Median=" +
StatUtil.median(d)
            + ", StdDev=" + StatUtil.stdDev(d));
    }
}
```

Output:

```
Sum      : [[6, 8], [10, 12]]
Product  : [[19, 22], [43, 50]]
Transpose: [[1, 3], [2, 4]]
Mean=5.5, Median=5.5, StdDev=1.707825127659933
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on creating reusable utility packages and using CLASSPATH has been implemented and verified.

Conclusion:

Thus, creating reusable utility packages and using CLASSPATH was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 14

Implement an Online Examination System using user-defined exceptions, exception propagation, and nested exception handling.

Aim:

To implement an Online Examination System that uses user-defined exceptions, exception propagation and nested exception handling.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A user-defined exception is created by extending Exception (checked) or RuntimeException (unchecked) and is thrown with the throw keyword. When a method does not handle an exception it is propagated up the call stack to the caller until some method catches it. Nested try-catch blocks allow an inner block to handle specific errors while unresolved errors pass to the outer block, and finally always executes for clean-up.

Procedure / Algorithm:

1. Create exceptions InvalidAnswerException and TimeExceededException.
2. Write answer() that validates the option and throws InvalidAnswerException.
3. Write attempt() that calls answer() and lets the exception propagate.
4. In main() use nested try-catch blocks with a finally block.
5. Run with valid, invalid and late answers and note the messages.

Code / Program:

```
class InvalidAnswerException extends Exception {
    InvalidAnswerException(String m) { super(m); }
}
class TimeExceededException extends RuntimeException {
    TimeExceededException(String m) { super(m); }
}
public class ExamSystem {
    static int score = 0;

    static void answer(int q, char opt, char correct, int sec) throws
InvalidAnswerException {
        if (sec > 60) throw new TimeExceededException("Q" + q + ": time exceeded");
        if (opt < 'A' || opt > 'D')
            throw new InvalidAnswerException("Q" + q + ": invalid option '" + opt +
"'");
        if (opt == correct) score++;
    }

    static void attempt(int q, char opt, char correct, int sec) throws
InvalidAnswerException {
        answer(q, opt, correct, sec);        // exception propagates to caller
        System.out.println("Q" + q + " recorded");
    }

    public static void main(String[] args) {
        char[] given = {'B', 'Z', 'C', 'A'};
        char[] key = {'B', 'A', 'C', 'D'};
        int[] time = {20, 15, 75, 30};
        try {
            for (int i = 0; i < given.length; i++) {
                try {
                    attempt(i + 1, given[i], key[i], time[i]);
                } catch (InvalidAnswerException e) {
                    System.out.println("Handled inner: " + e.getMessage());
                }
            }
        } catch (TimeExceededException e) {
```

```
            System.out.println("Handled outer: " + e.getMessage());
        } finally {
            System.out.println("Final score = " + score);
        }
    }
}
```

Output:

```
Q1 recorded
Handled inner: Q2: invalid option 'Z'
Handled outer: Q3: time exceeded
Final score = 1
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on user-defined exceptions and exception propagation has been implemented and verified.

Conclusion:

Thus, user-defined exceptions and exception propagation was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 15

Develop a multithreaded Railway Reservation System demonstrating thread priorities, synchronization, and thread life cycle.

Aim:

To develop a multithreaded Railway Reservation System that demonstrates thread priorities, synchronization and the thread life cycle.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A thread is a lightweight unit of execution created by extending Thread or implementing Runnable. Its life cycle has the states NEW, RUNNABLE, BLOCKED/WAITING/TIMED_WAITING and TERMINATED, which can be read with getState(). Priorities (1 to 10) are hints to the scheduler. When several threads book seats from shared data, a synchronized method allows only one thread at a time to enter, preventing race conditions and overbooking.

Procedure / Algorithm:

1. Create class Train with a synchronized bookSeat() method that reduces available seats.
2. Create Passenger threads that implement Runnable and call bookSeat().
3. Assign different priorities to the threads and start them.
4. Print getState() before start, while running and after join().
5. Verify that seats never become negative.

Code / Program:

```
class Train {
    private int seats = 3;
    synchronized void bookSeat(String who, int n) {
        if (seats >= n) {
            System.out.println(who + " booked " + n + " seat(s)");
            try { Thread.sleep(50); } catch (InterruptedException e) {}
            seats -= n;
        } else System.out.println(who + " - seats not available");
    }
    int left() { return seats; }
}
class Passenger extends Thread {
    Train t; int n;
    Passenger(Train t, String name, int n, int pr) {
        super(name); this.t = t; this.n = n; setPriority(pr);
    }
    public void run() { t.bookSeat(getName(), n); }
}
public class Railway {
    public static void main(String[] args) throws Exception {
        Train t = new Train();
        Passenger p1 = new Passenger(t, "Amit", 2, Thread.MAX_PRIORITY);
        Passenger p2 = new Passenger(t, "Neha", 2, Thread.NORM_PRIORITY);
        Passenger p3 = new Passenger(t, "Ravi", 1, Thread.MIN_PRIORITY);
        System.out.println("State before start: " + p1.getState());
        p1.start(); p2.start(); p3.start();
        System.out.println("State after start : " + p1.getState());
        p1.join(); p2.join(); p3.join();
        System.out.println("State after join  : " + p1.getState());
        System.out.println("Seats left = " + t.left());
    }
}
```

Output:

```
State before start: NEW
State after start : RUNNABLE
```

```
Amit booked 2 seat(s)
Ravi booked 1 seat(s)
Neha - seats not available
State after join  : TERMINATED
Seats left = 0
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on thread priorities, synchronization and thread life cycle has been implemented and verified.

Conclusion:

Thus, thread priorities, synchronization and thread life cycle was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 16

Implement the Producer–Consumer problem using multiple threads with wait(), notify(), and synchronization.

Aim:

To implement the Producer–Consumer problem using multiple threads with wait(), notify() and synchronization.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

The Producer–Consumer problem has producers that add items to a bounded buffer and consumers that remove them. Access to the buffer must be synchronized. If the buffer is full the producer calls wait() to release the monitor and sleep, and if the buffer is empty the consumer calls wait(); after changing the buffer a thread calls notifyAll() to wake the waiting threads. The wait() call is always placed in a while loop that re-checks the condition.

Procedure / Algorithm:

1. Create a Buffer class with a queue of capacity 3 and synchronized put() and take().
2. In put() wait while the buffer is full, add the item and call notifyAll().
3. In take() wait while the buffer is empty, remove the item and call notifyAll().
4. Create Producer and Consumer threads and start them.
5. Observe that production and consumption alternate without errors.

Code / Program:

```
import java.util.LinkedList;
import java.util.Queue;

class Buffer {
    private final Queue<Integer> q = new LinkedList<>();
    private final int cap = 3;
    synchronized void put(int v) throws InterruptedException {
        while (q.size() == cap) wait();          // buffer full
        q.add(v);
        System.out.println("Produced " + v + " " + q);
        notifyAll();
    }
    synchronized int take() throws InterruptedException {
        while (q.isEmpty()) wait();              // buffer empty
        int v = q.remove();
        System.out.println("      Consumed " + v + " " + q);
        notifyAll();
        return v;
    }
}
public class ProdCons {
    public static void main(String[] args) throws Exception {
        Buffer b = new Buffer();
        Thread p = new Thread(() -> {
            try { for (int i = 1; i <= 6; i++) { b.put(i); Thread.sleep(30); } }
            catch (InterruptedException e) {}
        });
        Thread c = new Thread(() -> {
            try { for (int i = 1; i <= 6; i++) { b.take(); Thread.sleep(80); } }
            catch (InterruptedException e) {}
        });
        p.start(); c.start();
        p.join(); c.join();
        System.out.println("Done");
    }
}
```

Output:

```
Produced 1 [1]
      Consumed 1 []
Produced 2 [2]
Produced 3 [2, 3]
      Consumed 2 [3]
Produced 4 [3, 4]
Produced 5 [3, 4, 5]
      Consumed 3 [4, 5]
Produced 6 [4, 5, 6]
      Consumed 4 [5, 6]
      Consumed 5 [6]
      Consumed 6 []
Done
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on inter-thread communication using wait() and notify() has been implemented and verified.

Conclusion:

Thus, inter-thread communication using wait() and notify() was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 17

Create a Java Applet to draw geometric shapes using mouse events and demonstrate the complete applet life cycle.

Aim:

To create a Java Applet that draws geometric shapes using mouse events and demonstrates the complete applet life cycle.

Software / Hardware Required:

JDK 8 (appletviewer support), text editor, Command Prompt / Terminal, web browser with applet support or appletviewer; PC with minimum 4 GB RAM.

Theory:

An applet is a Java program embedded in a web page and run by a browser or the appletviewer. Its life cycle methods are init() (one-time initialisation), start() (when the applet becomes visible), paint() (drawing), stop() (when the page is left) and destroy() (final clean-up). Mouse events are handled by implementing MouseListener and MouseMotionListener, and the shape is drawn with Graphics methods such as drawRect(), drawOval() and drawLine(). Note: the Applet API is deprecated, so JDK 8 is used with appletviewer.

Procedure / Algorithm:

1. Extend Applet and implement MouseListener; register it in init().
2. Print a message in init(), start(), stop() and destroy() to show the life cycle.
3. Store the click position in mousePressed() and call repaint().
4. In paint() draw a rectangle, oval and line at the stored position.
5. Compile and run with appletviewer ShapeApplet.html.

Code / Program:

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;
/* <applet code="ShapeApplet" width="400" height="300"></applet> */
public class ShapeApplet extends Applet implements MouseListener {
    int x = 50, y = 50, shape = 0;
    public void init() {
        addMouseListener(this);
        System.out.println("init() called");
    }
    public void start() { System.out.println("start() called"); }
    public void paint(Graphics g) {
        g.drawString("Click to draw shapes (" + x + "," + y + ")", 10, 15);
        if (shape == 0) g.drawRect(x, y, 80, 50);
        else if (shape == 1) g.drawOval(x, y, 80, 50);
        else g.drawLine(x, y, x + 80, y + 50);
    }
    public void mousePressed(MouseEvent e) {
        x = e.getX(); y = e.getY();
        shape = (shape + 1) % 3;
        repaint();
    }
    public void mouseClicked(MouseEvent e) {}
    public void mouseReleased(MouseEvent e) {}
    public void mouseEntered(MouseEvent e) {}
    public void mouseExited(MouseEvent e) {}
    public void stop() { System.out.println("stop() called"); }
    public void destroy() { System.out.println("destroy() called"); }
}
```

Output:

The applet window opens in appletviewer and shows a hint line. Every mouse click draws a rectangle, oval or line (in rotation) at the clicked position. The console prints init(), start(), stop() and destroy() when the applet is loaded, shown, minimised and closed.

Result:

The program was executed successfully and the expected output was obtained. The experiment on the applet life cycle and mouse event handling has been implemented and verified.

Conclusion:

Thus, the applet life cycle and mouse event handling was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 18

Develop a JDBC-based Student Management System implementing CRUD operations, transactions, and PreparedStatement.

Aim:

To develop a JDBC-based Student Management System that performs CRUD operations using PreparedStatement and transaction control.

Software / Hardware Required:

JDK 17 or later, MySQL Server 8.x, MySQL Connector/J (JDBC driver JAR), IDE (NetBeans / Eclipse / IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM.

Theory:

JDBC (Java Database Connectivity) is the standard API for connecting Java programs to relational databases. The steps are: load the driver, obtain a Connection with DriverManager, create a statement, execute SQL and close the resources. PreparedStatement precompiles the SQL and uses ? placeholders, which improves performance and prevents SQL injection. A transaction groups statements into one unit: setAutoCommit(false) starts it, commit() saves the changes and rollback() cancels them on failure.

Procedure / Algorithm:

1. Create database college and table student(id, name, course, marks) in MySQL.
2. Add the Connector/J JAR to the classpath and obtain a connection.
3. Insert, read, update and delete records using PreparedStatement.
4. Disable auto-commit, run the operations and call commit(); call rollback() in the catch block.
5. Display the table after each operation.

Code / Program:

```
import java.sql.*;

public class StudentJDBC {
    static final String URL = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root", PASS = "root";

    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {
            con.setAutoCommit(false);
            try {
                PreparedStatement ins = con.prepareStatement(
                    "INSERT INTO student(id,name,course,marks) VALUES(?,?,?,?)");
                ins.setInt(1, 1); ins.setString(2, "Rohit");
                ins.setString(3, "M.Tech"); ins.setInt(4, 85);
                ins.executeUpdate();

                PreparedStatement upd = con.prepareStatement(
                    "UPDATE student SET marks=? WHERE id=?");
                upd.setInt(1, 90); upd.setInt(2, 1);
                upd.executeUpdate();

                PreparedStatement del = con.prepareStatement(
                    "DELETE FROM student WHERE id=?");
                del.setInt(1, 99);
                del.executeUpdate();

                con.commit();
                System.out.println("Transaction committed");
            } catch (SQLException e) {
                con.rollback();
                System.out.println("Rolled back: " + e.getMessage());
            }
            ResultSet rs = con.createStatement().executeQuery("SELECT * FROM
student");
```

```
            while (rs.next())
                System.out.println(rs.getInt(1) + " " + rs.getString(2) + " "
                    + rs.getString(3) + " " + rs.getInt(4));
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
Transaction committed
1 Rohit M.Tech 90
(The record is inserted, its marks updated from 85 to 90 and the table is displayed.
If any statement fails, "Rolled back" is printed and the table stays unchanged.)
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on CRUD operations, PreparedStatement and transactions in JDBC has been implemented and verified.

Conclusion:

Thus, CRUD operations, PreparedStatement and transactions in JDBC was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 19

Create a multithreaded Banking System that performs synchronized database transactions using JDBC.

Aim:

To create a multithreaded Banking System that performs synchronized database transactions using JDBC.

Software / Hardware Required:

JDK 17 or later, MySQL Server 8.x, MySQL Connector/J (JDBC driver JAR), IDE (NetBeans / Eclipse / IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM.

Theory:

When several threads update the same account rows, race conditions can give wrong balances. Java synchronization (a synchronized method or block on a shared lock) lets only one thread execute the critical section at a time, while a JDBC transaction (setAutoCommit(false), commit(), rollback()) makes a transfer atomic so that either both the debit and the credit happen or neither does. Combining both gives thread-safe and consistent banking operations.

Procedure / Algorithm:

1. Create table account(acc_no, name, balance) in MySQL and insert two accounts.
2. Write a Bank class with a synchronized transfer() method using one transaction.
3. Debit one account, credit the other and commit; roll back on any error.
4. Create several Runnable tasks that call transfer() concurrently.
5. Display the final balances and check that the total is unchanged.

Code / Program:

```
import java.sql.*;

class Bank {
    static final String URL = "jdbc:mysql://localhost:3306/bank";
    synchronized void transfer(int from, int to, double amt) {
        try (Connection con = DriverManager.getConnection(URL, "root", "root")) {
            con.setAutoCommit(false);
            try (PreparedStatement d = con.prepareStatement(
                    "UPDATE account SET balance=balance-? WHERE acc_no=? AND
balance>=?");
                 PreparedStatement c = con.prepareStatement(
                    "UPDATE account SET balance=balance+? WHERE acc_no=?")) {
                d.setDouble(1, amt); d.setInt(2, from); d.setDouble(3, amt);
                if (d.executeUpdate() == 0) throw new SQLException("Insufficient
funds");
                c.setDouble(1, amt); c.setInt(2, to);
                c.executeUpdate();
                con.commit();
                System.out.println(Thread.currentThread().getName()
                    + ": transferred " + amt + " from " + from + " to " + to);
            } catch (SQLException e) {
                con.rollback();
                System.out.println(Thread.currentThread().getName() + ": " +
e.getMessage());
            }
        } catch (SQLException e) { e.printStackTrace(); }
    }
}
public class BankThreads {
    public static void main(String[] args) throws Exception {
        Bank bank = new Bank();
        Thread[] t = new Thread[4];
        for (int i = 0; i < t.length; i++) {
            t[i] = new Thread(() -> bank.transfer(101, 102, 500), "Client-" + (i +
1));
            t[i].start();
        }
```

```
        for (Thread x : t) x.join();
        System.out.println("All transactions finished");
    }
}
```

Output:

```
Client-1: transferred 500.0 from 101 to 102
Client-2: transferred 500.0 from 101 to 102
Client-3: transferred 500.0 from 101 to 102
Client-4: Insufficient funds
All transactions finished
(Output assumes account 101 starts with a balance of Rs. 1500; the transfers execute
one at a time and the total balance remains unchanged.)
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on synchronized threads combined with atomic JDBC transactions has been implemented and verified.

Conclusion:

Thus, synchronized threads combined with atomic JDBC transactions was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 20

Develop a Library Management System integrating inheritance, packages, exception handling, multithreading, interfaces, and JDBC.

Aim:

To develop a Library Management System that integrates inheritance, packages, exception handling, multithreading, interfaces and JDBC.

Software / Hardware Required:

JDK 17 or later, MySQL Server 8.x, MySQL Connector/J (JDBC driver JAR), IDE (NetBeans / Eclipse / IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM.

Theory:

A real application combines several Java features. An interface (Borrowable) defines the contract for issuing and returning items; a base class Item and a subclass Book use inheritance; classes are organised in the package library; a user-defined exception handles an unavailable book; threads allow several members to issue books at the same time, using synchronization; and JDBC stores the records in MySQL.

Procedure / Algorithm:

1. Create table book(id, title, author, available) in MySQL.
2. Create package library with interface Borrowable, class Item and subclass Book.
3. Create BookNotAvailableException for unavailable books.
4. Implement issueBook() and returnBook() with PreparedStatement in synchronized methods.
5. Start threads for several members issuing books and display the result.

Code / Program:

```
// FILE: library/LibraryApp.java
package library;
import java.sql.*;

interface Borrowable {
    void issue(int id) throws BookNotAvailableException, SQLException;
    void giveBack(int id) throws SQLException;
}
class BookNotAvailableException extends Exception {
    BookNotAvailableException(String m) { super(m); }
}
class Item {
    protected int id; protected String title;
    Item(int id, String title) { this.id = id; this.title = title; }
}
class Book extends Item implements Borrowable {
    static final String URL = "jdbc:mysql://localhost:3306/library";
    Book(int id, String title) { super(id, title); }
    public synchronized void issue(int id) throws BookNotAvailableException,
SQLException {
        try (Connection c = DriverManager.getConnection(URL, "root", "root");
             PreparedStatement p = c.prepareStatement(
               "UPDATE book SET available=0 WHERE id=? AND available=1")) {
            p.setInt(1, id);
            if (p.executeUpdate() == 0)
                throw new BookNotAvailableException("Book " + id + " not available");
        }
    }
    public synchronized void giveBack(int id) throws SQLException {
        try (Connection c = DriverManager.getConnection(URL, "root", "root");
             PreparedStatement p = c.prepareStatement(
               "UPDATE book SET available=1 WHERE id=?")) {
            p.setInt(1, id); p.executeUpdate();
        }
    }
}
public class LibraryApp {
```

```
    public static void main(String[] args) throws Exception {
        Book lib = new Book(1, "Java: The Complete Reference");
        for (int m = 1; m <= 3; m++) {
            final int member = m;
            new Thread(() -> {
                try { lib.issue(1); System.out.println("Member " + member + " issued
book 1"); }
                catch (BookNotAvailableException e) {
                    System.out.println("Member " + member + ": " + e.getMessage());
                } catch (SQLException e) { e.printStackTrace(); }
            }).start();
        }
    }
}
```

Output:

```
Member 1 issued book 1
Member 2: Book 1 not available
Member 3: Book 1 not available
(Only one member can issue the single available copy; the others receive
BookNotAvailableException.)
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on integration of OOP, exceptions, threads and JDBC has been implemented and verified.

Conclusion:

Thus, integration of OOP, exceptions, threads and JDBC was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 21

Develop a login system using Java Servlets that authenticates users, manages sessions, and restricts access to protected resources using deployment descriptors.

Aim:

To develop a login system using Java Servlets that authenticates users, manages sessions and restricts access to protected resources through the deployment descriptor.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, Servlet API 4.0 (servlet-api.jar), IDE (NetBeans / Eclipse), web browser (Chrome / Firefox); PC with minimum 4 GB RAM.

Theory:

A servlet is a Java class that runs inside a servlet container such as Tomcat and handles HTTP requests. Authentication is done by checking the submitted credentials, and HttpSession keeps the logged-in user across requests. The deployment descriptor web.xml maps URLs to servlets and can register a Filter for a URL pattern such as /secure/*, so every protected resource is checked centrally and unauthenticated users are redirected to the login page.

Procedure / Algorithm:

1. Create login.html with a form that posts username and password to LoginServlet.
2. In LoginServlet validate the credentials and store the user name in HttpSession.
3. Create AuthFilter that redirects to login.html if the session has no user.
4. Create ProfileServlet under /secure/profile and LogoutServlet that invalidates the session.
5. Register servlets, mappings and the filter in web.xml, deploy the WAR on Tomcat and test.

Code / Program:

```
// FILE: login.html
<form action="login" method="post">
  User: <input name="username"> Password: <input type="password" name="password">
  <input type="submit" value="Login">
</form>
// FILE: LoginServlet.java
import java.io.*; import javax.servlet.*; import javax.servlet.http.*;
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        String u = req.getParameter("username"), p = req.getParameter("password");
        if ("admin".equals(u) && "admin123".equals(p)) {
            req.getSession().setAttribute("user", u);
            res.sendRedirect("secure/profile");
        } else res.getWriter().println("Invalid login. <a
href='login.html'>Retry</a>");
    }
}
// FILE: AuthFilter.java
import java.io.*; import javax.servlet.*; import javax.servlet.http.*;
public class AuthFilter implements Filter {
    public void doFilter(ServletRequest rq, ServletResponse rs, FilterChain ch)
            throws IOException, ServletException {
        HttpSession s = ((HttpServletRequest) rq).getSession(false);
        if (s != null && s.getAttribute("user") != null) ch.doFilter(rq, rs);
        else ((HttpServletResponse) rs).sendRedirect("../login.html");
    }
}
// FILE: ProfileServlet.java
import java.io.*; import javax.servlet.*; import javax.servlet.http.*;
public class ProfileServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        res.setContentType("text/html");
        res.getWriter().println("Welcome, " + req.getSession().getAttribute("user")
            + " <a href='../logout'>Logout</a>");
```

```
    }
}
// FILE: LogoutServlet.java
import java.io.*; import javax.servlet.*; import javax.servlet.http.*;
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        req.getSession().invalidate();
        res.sendRedirect("login.html");
    }
}
// FILE: WEB-INF/web.xml
<web-app>
 <servlet><servlet-name>L</servlet-name><servlet-class>LoginServlet</servlet-class></
servlet>
 <servlet-mapping><servlet-name>L</servlet-name><url-pattern>/login</url-pattern></
servlet-mapping>
 <servlet><servlet-name>P</servlet-name><servlet-class>ProfileServlet</servlet-
class></servlet>
 <servlet-mapping><servlet-name>P</servlet-name><url-pattern>/secure/profile</url-
pattern></servlet-mapping>
 <servlet><servlet-name>O</servlet-name><servlet-class>LogoutServlet</servlet-
class></servlet>
 <servlet-mapping><servlet-name>O</servlet-name><url-pattern>/logout</url-pattern></
servlet-mapping>
 <filter><filter-name>A</filter-name><filter-class>AuthFilter</filter-class></filter>
 <filter-mapping><filter-name>A</filter-name><url-pattern>/secure/*</url-pattern></
filter-mapping>
</web-app>
```

Output:

The login page is displayed. Entering admin / admin123 redirects to /secure/profile showing "Welcome, admin". Wrong credentials show "Invalid login", and opening /secure/profile without logging in redirects to the login page.

Result:

The program was executed successfully and the expected output was obtained. The experiment on authentication, sessions and deployment-descriptor based access control has been implemented and verified.

Conclusion:

Thus, authentication, sessions and deployment-descriptor based access control was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 22

Create a multi-module web application with multiple servlets configured entirely through web.xml, demonstrating servlet mapping, initialization parameters, and load-on-startup.

Aim:

To create a web application with multiple servlets configured entirely through web.xml, demonstrating servlet mapping, initialization parameters and load-on-startup.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, Servlet API 4.0 (servlet-api.jar), IDE (NetBeans / Eclipse), web browser (Chrome / Firefox); PC with minimum 4 GB RAM.

Theory:

web.xml is the deployment descriptor of a web application. The <servlet> element declares a servlet class, and <servlet-mapping> binds it to a URL pattern. Initialization parameters (<init-param>) give a servlet configurable values read through getInitParameter() in init() via ServletConfig, whereas <context-param> values are shared by the whole application through ServletContext. The <load-on-startup> element with a positive integer makes the container create and initialise the servlet at deployment, in ascending order of the number, instead of at the first request.

Procedure / Algorithm:

1. Create three servlets: HomeServlet, ConfigServlet and ReportServlet (one per module).
2. Read init-parameters with getServletConfig().getInitParameter() and a context-parameter with getServletContext().
3. Declare each servlet in web.xml with a servlet-mapping.
4. Set load-on-startup values 1, 2 and 3.
5. Deploy on Tomcat, check the console for initialisation order and open each URL.

Code / Program:

```
// FILE: ConfigServlet.java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ConfigServlet extends HttpServlet {
    String module;
    public void init() {
        module = getServletConfig().getInitParameter("module");
        System.out.println("ConfigServlet initialised for module " + module);
    }
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h3>Module: " + module + "</h3>");
        out.println("App name: " + getServletContext().getInitParameter("appName"));
    }
}
// FILE: HomeServlet.java  (ReportServlet is written the same way)
public class HomeServlet extends HttpServlet {
    public void init() { System.out.println("HomeServlet initialised"); }
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        res.setContentType("text/html");
        res.getWriter().println("<h2>Home module</h2>");
    }
}
// FILE: WEB-INF/web.xml
<web-app>
  <context-param>
    <param-name>appName</param-name><param-value>Multi Module App</param-value>
  </context-param>
```

```
  <servlet>
    <servlet-name>Home</servlet-name><servlet-class>HomeServlet</servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet>
    <servlet-name>Config</servlet-name><servlet-class>ConfigServlet</servlet-class>
    <init-param><param-name>module</param-name><param-value>Admin</param-value></init-
param>
    <load-on-startup>2</load-on-startup>
  </servlet>
  <servlet>
    <servlet-name>Report</servlet-name><servlet-class>ReportServlet</servlet-class>
    <load-on-startup>3</load-on-startup>
  </servlet>
  <servlet-mapping><servlet-name>Home</servlet-name><url-pattern>/home</url-
pattern></servlet-mapping>
  <servlet-mapping><servlet-name>Config</servlet-name><url-pattern>/config</url-
pattern></servlet-mapping>
  <servlet-mapping><servlet-name>Report</servlet-name><url-pattern>/report</url-
pattern></servlet-mapping>
</web-app>
```

Output:

On deployment the Tomcat console prints the initialisation messages in the order Home, Config, Report. The URL /config displays "Module: Admin" and "App name: Multi Module App"; /home and /report display their own module pages.

Result:

The program was executed successfully and the expected output was obtained. The experiment on servlet mapping, init parameters and load-on-startup has been implemented and verified.

Conclusion:

Thus, servlet mapping, init parameters and load-on-startup was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 23

Design an online examination system using Servlets that processes dynamic forms, validates user input, maintains session state, and generates a result report.

Aim:

To design an online examination system using Servlets that processes dynamic forms, validates input, maintains session state and generates a result report.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, Servlet API 4.0 (servlet-api.jar), IDE (NetBeans / Eclipse), web browser (Chrome / Firefox); PC with minimum 4 GB RAM.

Theory:

A servlet can generate an HTML form dynamically from data such as a question bank. Submitted values are read with request.getParameter() and validated on the server (null or empty checks, allowed options). HttpSession stores the candidate name and answers so that the state is kept between the form request and the result request. After evaluation the servlet writes a result report with score, percentage and grade.

Procedure / Algorithm:

1. Create a question bank (questions, options and correct answers) inside ExamServlet.
2. In doGet() read the candidate name, store it in the session and generate the question form.
3. In doPost() validate that every question is answered.
4. Compare the answers with the key, compute the score and percentage.
5. Print the result report using the data stored in the session.

Code / Program:

```
// FILE: ExamServlet.java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ExamServlet extends HttpServlet {
    String[] q = {"Which keyword is used to inherit a class?",
                  "Which method starts a thread?",
                  "Default port of Tomcat?"};
    String[][] opt = {{"extends", "implements", "inherits"},
                      {"run()", "start()", "begin()"},
                      {"80", "8080", "3306"}};
    int[] key = {0, 1, 1};

    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        String name = req.getParameter("name");
        if (name == null || name.trim().isEmpty()) name = "Candidate";
        req.getSession().setAttribute("name", name);
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h3>Online Exam - " + name + "</h3><form method='post'>");
        for (int i = 0; i < q.length; i++) {
            out.println("<p>" + (i + 1) + ". " + q[i] + "<br>");
            for (int j = 0; j < 3; j++)
                out.println("<input type='radio' name='q" + i + "' value='" + j + "'>"
                    + opt[i][j] + " ");
            out.println("</p>");
        }
        out.println("<input type='submit' value='Submit'></form>");
    }

    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        int score = 0;
        for (int i = 0; i < q.length; i++) {
```

```
            String a = req.getParameter("q" + i);
            if (a == null) {
                out.println("Please answer question " + (i + 1)); return;   //
validation
            }
            if (Integer.parseInt(a) == key[i]) score++;
        }
        String name = (String) req.getSession().getAttribute("name");
        double pct = score * 100.0 / q.length;
        out.println("<h3>Result Report</h3>Name: " + name + "<br>Score: " + score +
"/"
            + q.length + "<br>Percentage: " + pct + "%<br>Grade: " + (pct >= 60 ?
"Pass" : "Fail"));
    }
}
```

Output:

Opening /exam?name=Rohit shows the question form with radio buttons. Submitting with an unanswered question shows a validation message; a complete submission shows the Result Report with name, score (for example 3/3), percentage and Pass/Fail grade.

Result:

The program was executed successfully and the expected output was obtained. The experiment on dynamic form generation, validation and session state in servlets has been implemented and verified.

Conclusion:

Thus, dynamic form generation, validation and session state in servlets was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 24

Develop a shopping cart application using Servlets that demonstrates all session tracking techniques (Cookies, URL Rewriting, Hidden Fields, and HttpSession) and compares their behaviour.

Aim:

To develop a shopping cart using Servlets that demonstrates all four session tracking techniques (Cookies, URL Rewriting, Hidden Fields and HttpSession) and compares their behaviour.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, Servlet API 4.0 (servlet-api.jar), IDE (NetBeans / Eclipse), web browser (Chrome / Firefox); PC with minimum 4 GB RAM.

Theory:

HTTP is stateless, so a server must track a user across requests. Cookies store small data on the client and survive browser restarts if given an expiry, but can be disabled. URL rewriting appends the data (or the session id) to the link, works without cookies but exposes data in the address bar. Hidden form fields carry data inside the form and work only for form submissions. HttpSession keeps data on the server and uses a session id (usually a cookie) to identify the client; it is the most secure and flexible technique, but uses server memory.

Procedure / Algorithm:

1. Create CartServlet that takes the parameters mode (cookie, url, hidden, session) and item.
2. For cookie mode add a Cookie named lastItem and read it on the next request.
3. For URL rewriting append ?item= to the link and use response.encodeURL().
4. For hidden mode generate a form with <input type='hidden'> carrying the item count.
5. For session mode store the item list in HttpSession; display the comparison table.

Code / Program:

```
// FILE: CartServlet.java
import java.io.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class CartServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        String mode = req.getParameter("mode"), item = req.getParameter("item");
        if (mode == null) mode = "session";
        if (item == null) item = "Pen";

        if (mode.equals("cookie")) {
            String last = "none";
            for (Cookie c : req.getCookies() == null ? new Cookie[0] :
req.getCookies())
                if (c.getName().equals("lastItem")) last = c.getValue();
            Cookie c = new Cookie("lastItem", item);
            c.setMaxAge(3600);
            res.addCookie(c);
            out.println("Cookie: previous item = " + last + ", current = " + item);
        } else if (mode.equals("url")) {
            String link = res.encodeURL("cart?mode=url&item=Book");
            out.println("URL rewriting: item = " + item + "<br><a href='" + link +
"'>Add Book</a>");
        } else if (mode.equals("hidden")) {
            String n = req.getParameter("count");
            int count = (n == null) ? 1 : Integer.parseInt(n) + 1;
            out.println("Hidden field: items so far = " + count);
            out.println("<form><input type='hidden' name='mode' value='hidden'>"
                + "<input type='hidden' name='count' value='" + count + "'>"
```

```
                + "<input type='submit' value='Add item'></form>");
        } else {
            HttpSession s = req.getSession();
            List<String> cart = (List<String>) s.getAttribute("cart");
            if (cart == null) cart = new ArrayList<>();
            cart.add(item);
            s.setAttribute("cart", cart);
            out.println("HttpSession: cart = " + cart + ", id = " + s.getId());
        }
        out.println("<hr>Comparison: Cookie (client, persistent), URL (client,
visible), "
            + "Hidden (client, forms only), HttpSession (server, secure)");
    }
}
```

Output:

/cart?mode=cookie shows the previous and current item from the cookie; mode=url shows the rewritten link; mode=hidden increments the item count carried by the hidden field; mode=session shows the cart list and the session id. Each page ends with the comparison line.

Result:

The program was executed successfully and the expected output was obtained. The experiment on cookies, URL rewriting, hidden fields and HttpSession for session tracking has been implemented and verified.

Conclusion:

Thus, cookies, URL rewriting, hidden fields and HttpSession for session tracking was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 25

Implement a file upload and download system using Servlets that validates uploaded files, stores metadata, and handles exceptions gracefully.

Aim:

To implement a file upload and download system using Servlets that validates the uploaded files, stores metadata and handles exceptions gracefully.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, Servlet API 4.0 (servlet-api.jar), IDE (NetBeans / Eclipse), web browser (Chrome / Firefox); PC with minimum 4 GB RAM.

Theory:

Servlet 3.0 supports file upload with the @MultipartConfig annotation, and the uploaded part is read with request.getPart(). The file name, size and content type are the metadata of the file, and validation checks the extension, size limit and empty uploads. A download servlet sets the Content-Disposition header to attachment and streams the file bytes to the client. Exceptions such as IOException or ServletException are caught and a friendly message is shown instead of a stack trace.

Procedure / Algorithm:

1. Create upload.html with a multipart/form-data form containing a file input.
2. Annotate UploadServlet with @MultipartConfig and a maximum file size.
3. Validate extension (.pdf, .txt, .jpg) and size, then save the file to the upload folder.
4. Store file name, size and type in a list and display it with download links.
5. Write DownloadServlet that streams the selected file; catch exceptions and show messages.

Code / Program:

```
// FILE: UploadServlet.java
import java.io.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.annotation.*;
import javax.servlet.http.*;
@WebServlet("/upload")
@MultipartConfig(maxFileSize = 2 * 1024 * 1024)
public class UploadServlet extends HttpServlet {
    static List<String> meta = new ArrayList<>();
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        try {
            Part p = req.getPart("file");
            String name = new File(p.getSubmittedFileName()).getName();
            if (name.isEmpty() || p.getSize() == 0) { out.println("No file selected");
return; }
            if (!name.matches("(?i).+\\.(pdf|txt|jpg)")) { out.println("Invalid file
type"); return; }
            File dir = new File(getServletContext().getRealPath("/uploads"));
            if (!dir.exists()) dir.mkdirs();
            p.write(dir.getAbsolutePath() + File.separator + name);
            meta.add(name + " | " + p.getSize() + " bytes | " + p.getContentType());
            out.println("<h3>Uploaded files</h3>");
            for (String m : meta) {
                String f = m.split(" \\| ")[0];
                out.println(m + " - <a href='download?file=" + f +
"'>Download</a><br>");
            }
        } catch (IllegalStateException e) {
            out.println("File too large (max 2 MB)");
        } catch (Exception e) {
            out.println("Upload failed: " + e.getMessage());
        }
```

```
    }
}
// FILE: DownloadServlet.java
@WebServlet("/download")
public class DownloadServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        String name = new File(req.getParameter("file")).getName();
        File f = new File(getServletContext().getRealPath("/uploads"), name);
        if (!f.exists()) { res.sendError(404, "File not found"); return; }
        res.setContentType("application/octet-stream");
        res.setHeader("Content-Disposition", "attachment; filename=\"" + name + "\"");
        try (InputStream in = new FileInputStream(f); OutputStream o =
res.getOutputStream()) {
            in.transferTo(o);
        }
    }
}
// FILE: upload.html
<form action="upload" method="post" enctype="multipart/form-data">
  <input type="file" name="file"> <input type="submit" value="Upload">
</form>
```

Output:

A valid .pdf/.txt/.jpg file is saved and the metadata list (name, size, type) is shown with a Download link. Invalid types, empty selections and files above 2 MB produce the corresponding error messages; clicking Download saves the file.

Result:

The program was executed successfully and the expected output was obtained. The experiment on file upload, download and metadata handling in servlets has been implemented and verified.

Conclusion:

Thus, file upload, download and metadata handling in servlets was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 26

Create a servlet-based Employee Management System implementing CRUD operations with HTML forms, request dispatching, and configurable servlet initialization parameters.

Aim:

To create a servlet-based Employee Management System that implements CRUD operations with HTML forms, request dispatching and configurable initialization parameters.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, Servlet / JSP API, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

CRUD stands for Create, Read, Update and Delete, the four basic database operations. A servlet receives the HTML form data, performs the JDBC operation with PreparedStatement and then shows the result. Request dispatching (RequestDispatcher.forward() or include()) transfers control to another resource on the server without involving the browser. The database URL, user and password are kept in <init-param> elements of web.xml so that they can be changed without recompiling.

Procedure / Algorithm:

1. Create table employee(id, name, dept, salary) in MySQL.
2. Declare dbUrl, dbUser and dbPass as init-params of EmployeeServlet in web.xml.
3. Create employee.html with a form and an action field (add, update, delete, list).
4. Execute the matching SQL with PreparedStatement based on the action.
5. Forward to ListServlet using RequestDispatcher to display all employees.

Code / Program:

```
// FILE: employee.html
<form action="employee" method="post">
  ID <input name="id"> Name <input name="name"> Dept <input name="dept"> Salary <input
name="salary">
  <select name="action"><option>add</option><option>update</option><option>delete</
option></select>
  <input type="submit">
</form>
// FILE: EmployeeServlet.java
import java.io.*; import java.sql.*; import javax.servlet.*; import
javax.servlet.http.*;
public class EmployeeServlet extends HttpServlet {
    String url, user, pass;
    public void init() throws ServletException {            // init-params from
web.xml
        url = getInitParameter("dbUrl"); user = getInitParameter("dbUser"); pass =
getInitParameter("dbPass");
        try { Class.forName("com.mysql.cj.jdbc.Driver"); } catch (Exception e) { throw
new ServletException(e); }
    }
    protected void doPost(HttpServletRequest rq, HttpServletResponse rs) throws
ServletException, IOException {
        String a = rq.getParameter("action"), id = rq.getParameter("id");
        try (Connection c = DriverManager.getConnection(url, user, pass)) {
            PreparedStatement p;
            if (a.equals("add")) {
                p = c.prepareStatement("INSERT INTO employee VALUES(?,?,?,?)");
                p.setString(1, id); p.setString(2, rq.getParameter("name"));
                p.setString(3, rq.getParameter("dept")); p.setString(4,
rq.getParameter("salary"));
            } else if (a.equals("update")) {
                p = c.prepareStatement("UPDATE employee SET salary=? WHERE id=?");
                p.setString(1, rq.getParameter("salary")); p.setString(2, id);
            } else {
```

```
                p = c.prepareStatement("DELETE FROM employee WHERE id=?");
p.setString(1, id);
            }
            p.executeUpdate();
        } catch (SQLException e) { throw new ServletException(e); }
        rq.getRequestDispatcher("list").forward(rq, rs);     // request dispatching
    }
}
// FILE: ListServlet.java
import java.io.*; import java.sql.*; import javax.servlet.*; import
javax.servlet.http.*;
public class ListServlet extends HttpServlet {
    protected void service(HttpServletRequest rq, HttpServletResponse rs) throws
IOException {
        PrintWriter out = rs.getWriter();
        out.println("<table border=1>");
        try (Connection c =
DriverManager.getConnection(getServletContext().getInitParameter("dbUrl"), "root",
"root");
             ResultSet r = c.createStatement().executeQuery("SELECT * FROM employee"))
{
            while (r.next()) out.println("<tr><td>" + r.getString(1) + "</td><td>" +
r.getString(2)
                + "</td><td>" + r.getString(3) + "</td><td>" + r.getString(4) +
"</td></tr>");
        } catch (SQLException e) { out.println(e.getMessage()); }
        out.println("</table><a href='employee.html'>Back</a>");
    }
}
// FILE: WEB-INF/web.xml
<web-app>
 <context-param><param-name>dbUrl</param-name><param-value>jdbc:mysql://
localhost:3306/company</param-value></context-param>
 <servlet><servlet-name>E</servlet-name><servlet-class>EmployeeServlet</servlet-class>
  <init-param><param-name>dbUrl</param-name><param-value>jdbc:mysql://localhost:3306/
company</param-value></init-param>
  <init-param><param-name>dbUser</param-name><param-value>root</param-value></init-
param>
  <init-param><param-name>dbPass</param-name><param-value>root</param-value></init-
param></servlet>
 <servlet-mapping><servlet-name>E</servlet-name><url-pattern>/employee</url-
pattern></servlet-mapping>
 <servlet><servlet-name>L</servlet-name><servlet-class>ListServlet</servlet-class></
servlet>
 <servlet-mapping><servlet-name>L</servlet-name><url-pattern>/list</url-pattern></
servlet-mapping>
</web-app>
```

Output:

The HTML form is used to add, update or delete an employee; after every operation the request is forwarded to ListServlet, which displays the employee table with the changes reflected.

Result:

The program was executed successfully and the expected output was obtained. The experiment on CRUD with servlets, request dispatching and init parameters has been implemented and verified.

Conclusion:

Thus, CRUD with servlets, request dispatching and init parameters was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 27

Develop a secure online banking portal using GenericServlet and HttpServlet to demonstrate authentication, authorization, session timeout, and servlet life cycle methods.

Aim:

To develop a secure online banking portal using GenericServlet and HttpServlet that demonstrates authentication, authorization, session timeout and servlet life cycle methods.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, Servlet API 4.0 (servlet-api.jar), IDE (NetBeans / Eclipse), web browser (Chrome / Firefox); PC with minimum 4 GB RAM.

Theory:

GenericServlet is a protocol-independent servlet whose service() method handles every request, while HttpServlet adds HTTP-specific methods such as doGet() and doPost(). The life cycle of a servlet consists of init() (called once), service() (called for every request) and destroy() (called once at unloading). Authentication verifies who the user is, authorization decides what the user may do (here, a role check for the manager page), and session timeout invalidates idle sessions using setMaxInactiveInterval() or <session-timeout> in web.xml.

Procedure / Algorithm:

1. Create BankInfo extending GenericServlet and print life cycle messages in init(), service() and destroy().
2. Create BankLogin extending HttpServlet that authenticates and stores user, role and a 60-second inactivity limit in the session.
3. Create AccountServlet that allows only logged-in users and ManagerServlet that also requires role manager.
4. Redirect unauthenticated users to login and show a message when the session has expired.
5. Declare the servlets and <session-config> in web.xml and test on Tomcat.

Code / Program:

```
// FILE: BankInfo.java
import java.io.*;
import javax.servlet.*;
public class BankInfo extends GenericServlet {
    public void init() { System.out.println("BankInfo: init() called"); }
    public void service(ServletRequest rq, ServletResponse rs) throws IOException {
        rs.setContentType("text/html");
        rs.getWriter().println("<h3>Secure Bank</h3>Open 10 AM - 4 PM. Login to
continue.");
        System.out.println("BankInfo: service() called");
    }
    public void destroy() { System.out.println("BankInfo: destroy() called"); }
}
// FILE: BankLogin.java
import java.io.*;
import java.util.*;
import javax.servlet.http.*;
public class BankLogin extends HttpServlet {
    Map<String, String[]> users = new HashMap<>();   // user -> {password, role}
    public void init() {
        users.put("rohit", new String[]{"pass1", "customer"});
        users.put("manager", new String[]{"pass2", "manager"});
    }
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        String u = req.getParameter("user"), p = req.getParameter("pass");
        String[] d = users.get(u);
        if (d != null && d[0].equals(p)) {
            HttpSession s = req.getSession();
```

```
            s.setAttribute("user", u);
            s.setAttribute("role", d[1]);
            s.setMaxInactiveInterval(60);               // session timeout
            res.sendRedirect("account");
        } else res.sendError(401, "Authentication failed");
    }
}
// FILE: AccountServlet.java
public class AccountServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        HttpSession s = req.getSession(false);
        res.setContentType("text/html");
        if (s == null || s.getAttribute("user") == null) {
            res.getWriter().println("Session expired or not logged in. <a
href='login.html'>Login</a>");
            return;
        }
        String role = (String) s.getAttribute("role");
        res.getWriter().println("Welcome " + s.getAttribute("user") + " (" + role +
")");
        if (!role.equals("manager")) res.getWriter().println("<br>Manager page: access
denied");
        else res.getWriter().println("<br>Manager page: all accounts report");
    }
}
// FILE: WEB-INF/web.xml (relevant part)
<session-config><session-timeout>1</session-timeout></session-config>
```

Output:

/bankinfo prints the public message and the console shows init(), service() and destroy() messages. Logging in as rohit shows "Welcome rohit (customer)" with manager access denied; manager gets the manager report. After 60 seconds of inactivity the portal shows "Session expired".

Result:

The program was executed successfully and the expected output was obtained. The experiment on authentication, authorization, session timeout and the servlet life cycle has been implemented and verified.

Conclusion:

Thus, authentication, authorization, session timeout and the servlet life cycle was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 28

Build a visitor analytics application that records client requests, browser information, session duration, and page visits using Servlet APIs and deployment descriptor configuration.

Aim:

To build a visitor analytics application that records client requests, browser information, session duration and page visits using Servlet APIs and deployment descriptor configuration.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, Servlet API 4.0 (servlet-api.jar), IDE (NetBeans / Eclipse), web browser (Chrome / Firefox); PC with minimum 4 GB RAM.

Theory:

The Servlet API provides request information such as the client IP address (getRemoteAddr()), the browser (the User-Agent header) and the requested URI. Listeners such as HttpSessionListener are notified when a session is created or destroyed, so the session duration can be calculated from the creation and last accessed times. Listeners, filters and servlets are all registered in web.xml. An AtomicInteger or a synchronized counter stored in the ServletContext keeps total page visits safely for concurrent users.

Procedure / Algorithm:

1. Create AnalyticsFilter that counts page visits and logs IP, URI and User-Agent for every request.
2. Create SessionTracker implementing HttpSessionListener to record creation and destruction times and the duration.
3. Create StatsServlet that displays the totals, active sessions and the last requests.
4. Register the filter, listener and servlet in web.xml.
5. Open different pages from different browsers and view /stats.

Code / Program:

```
// FILE: AnalyticsFilter.java
import java.io.*;
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import javax.servlet.*;
import javax.servlet.http.*;
public class AnalyticsFilter implements Filter {
    static AtomicInteger visits = new AtomicInteger();
    static List<String> log = Collections.synchronizedList(new ArrayList<>());
    public void doFilter(ServletRequest rq, ServletResponse rs, FilterChain ch)
            throws IOException, ServletException {
        HttpServletRequest r = (HttpServletRequest) rq;
        visits.incrementAndGet();
        log.add(new Date() + " | " + r.getRemoteAddr() + " | " + r.getRequestURI()
                + " | " + r.getHeader("User-Agent"));
        r.getSession();                    // make sure a session exists
        ch.doFilter(rq, rs);
    }
}
// FILE: SessionTracker.java
public class SessionTracker implements HttpSessionListener {
    static AtomicInteger active = new AtomicInteger();
    public void sessionCreated(HttpSessionEvent e) {
        active.incrementAndGet();
    }
    public void sessionDestroyed(HttpSessionEvent e) {
        active.decrementAndGet();
        HttpSession s = e.getSession();
        long sec = (s.getLastAccessedTime() - s.getCreationTime()) / 1000;
        System.out.println("Session " + s.getId() + " lasted " + sec + " s");
    }
}
// FILE: StatsServlet.java
```

```
public class StatsServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        HttpSession s = req.getSession();
        long dur = (System.currentTimeMillis() - s.getCreationTime()) / 1000;
        out.println("<h3>Visitor Analytics</h3>Total page visits: " +
AnalyticsFilter.visits
            + "<br>Active sessions: " + SessionTracker.active
            + "<br>Your session duration: " + dur + " s<br><pre>");
        for (String l : AnalyticsFilter.log) out.println(l);
        out.println("</pre>");
    }
}
// FILE: WEB-INF/web.xml
<web-app>
  <filter><filter-name>An</filter-name><filter-class>AnalyticsFilter</filter-class></
filter>
  <filter-mapping><filter-name>An</filter-name><url-pattern>/*</url-pattern></filter-
mapping>
  <listener><listener-class>SessionTracker</listener-class></listener>
  <servlet><servlet-name>Stats</servlet-name><servlet-class>StatsServlet</servlet-
class></servlet>
  <servlet-mapping><servlet-name>Stats</servlet-name><url-pattern>/stats</url-
pattern></servlet-mapping>
  <session-config><session-timeout>5</session-timeout></session-config>
</web-app>
```

Output:

/stats shows the total page visits, number of active sessions, the duration of the current session and a log line for every request containing the time, client IP, requested URI and browser (User-Agent). The console prints the duration of each session when it expires.

Result:

The program was executed successfully and the expected output was obtained. The experiment on filters, listeners and the servlet API for visitor analytics has been implemented and verified.

Conclusion:

Thus, filters, listeners and the servlet API for visitor analytics was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 29

Create a servlet application that dynamically generates reports based on user input, uses request forwarding and redirection, and manages concurrent client requests efficiently.

Aim:

To create a servlet application that generates reports dynamically from user input, uses request forwarding and redirection, and manages concurrent client requests efficiently.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, Servlet API 4.0 (servlet-api.jar), IDE (NetBeans / Eclipse), web browser (Chrome / Firefox); PC with minimum 4 GB RAM.

Theory:

A report servlet reads the criteria entered by the user and builds an HTML report dynamically. Forwarding (RequestDispatcher.forward()) passes the same request to another server resource, keeps the URL unchanged and shares request attributes, while redirection (sendRedirect()) tells the browser to send a new request to a different URL. A servlet instance is shared by all clients, so instance variables are not thread-safe; local variables, synchronized blocks and atomic classes are used so that concurrent requests do not interfere.

Procedure / Algorithm:

1. Create report.html with fields for the report type and number of records.
2. In ReportServlet validate the input; on error redirect to report.html?error=1.
3. Generate the data, store it as a request attribute and forward to ReportView.
4. Use a local variable for report data and an AtomicInteger for the number of reports served.
5. Test concurrent access using several browser tabs.

Code / Program:

```
// FILE: ReportServlet.java
import java.io.*;
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;
import javax.servlet.*;
import javax.servlet.http.*;
public class ReportServlet extends HttpServlet {
    private final AtomicInteger served = new AtomicInteger();   // shared, thread-safe

    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        String type = req.getParameter("type");
        int n;
        try { n = Integer.parseInt(req.getParameter("count")); }
        catch (NumberFormatException e) { n = -1; }
        if (type == null || n <= 0 || n > 100) {
            res.sendRedirect("report.html?error=1");           // redirection
            return;
        }
        List<String> rows = new ArrayList<>();                  // local = thread-safe
        for (int i = 1; i <= n; i++)
            rows.add(type + " record " + i + " - value " + (i * 10));
        req.setAttribute("rows", rows);
        req.setAttribute("title", type + " Report");
        req.setAttribute("served", served.incrementAndGet());
        req.getRequestDispatcher("view").forward(req, res);     // forwarding
    }
}
// FILE: ReportView.java
public class ReportView extends HttpServlet {
    protected void service(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
```

```
        out.println("<h3>" + req.getAttribute("title") + "</h3><ol>");
        for (String r : (List<String>) req.getAttribute("rows")) out.println("<li>" +
r + "</li>");
        out.println("</ol>Reports served so far: " + req.getAttribute("served"));
    }
}
// FILE: report.html
<form action="report" method="post">
  Type: <select name="type"><option>Sales</option><option>Stock</option></select>
  Records: <input name="count"> <input type="submit" value="Generate">
</form>
```

Output:

Selecting Sales with 5 records forwards to /view (URL stays /report) and displays a numbered Sales Report with the count of reports served. Invalid input redirects the browser to report.html?error=1 (URL changes).

Result:

The program was executed successfully and the expected output was obtained. The experiment on forward versus redirect and thread-safe servlet design has been implemented and verified.

Conclusion:

Thus, forward versus redirect and thread-safe servlet design was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 30

Develop a complete e-commerce web application using Servlets, deployment descriptors, session management, and servlet life cycle methods, with configurable initialization parameters defined in web.xml.

Aim:

To develop a complete e-commerce web application using Servlets, deployment descriptors, session management and life cycle methods, with initialization parameters configured in web.xml.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, Servlet API 4.0 (servlet-api.jar), IDE (NetBeans / Eclipse), web browser (Chrome / Firefox); PC with minimum 4 GB RAM.

Theory:

An e-commerce application consists of a product catalogue, a shopping cart and a checkout. The catalogue is loaded once in init() of the ProductServlet (life cycle), shop name and tax rate come from <init-param> and <context-param> entries in web.xml, and the cart is stored in HttpSession so that each customer has a separate cart. CheckoutServlet calculates the total with tax, shows the bill and invalidates the session. destroy() releases resources when the application is stopped.

Procedure / Algorithm:

1. Define shopName, taxRate and currency as parameters in web.xml.
2. In ProductServlet.init() load the product list and read the parameters.
3. Add-to-cart requests store product id and quantity in the session cart.
4. CheckoutServlet calculates subtotal, tax and total and prints the bill.
5. Invalidate the session after checkout and test the complete flow.

Code / Program:

```
// FILE: ShopServlet.java
import java.io.*;
import java.util.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ShopServlet extends HttpServlet {
    Map<Integer, String> names = new LinkedHashMap<>();
    Map<Integer, Double> price = new HashMap<>();
    String shop;
    public void init() {
        shop = getInitParameter("shopName");
        names.put(1, "Keyboard"); price.put(1, 799.0);
        names.put(2, "Mouse");    price.put(2, 399.0);
        names.put(3, "Monitor");  price.put(3, 8999.0);
        System.out.println(shop + " catalogue loaded");
    }
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        String add = req.getParameter("add");
        HttpSession s = req.getSession();
        Map<Integer, Integer> cart = (Map<Integer, Integer>) s.getAttribute("cart");
        if (cart == null) { cart = new HashMap<>(); s.setAttribute("cart", cart); }
        if (add != null) cart.merge(Integer.parseInt(add), 1, Integer::sum);
        out.println("<h2>" + shop + "</h2>");
        for (int id : names.keySet())
            out.println(names.get(id) + " - Rs. " + price.get(id)
                + " <a href='shop?add=" + id + "'>Add to cart</a><br>");
        out.println("<br>Items in cart: " + cart.values().stream().mapToInt(i ->
i).sum()
            + " | <a href='checkout'>Checkout</a>");
    }
    public void destroy() { System.out.println(shop + " stopped"); }
```

```
}
// FILE: CheckoutServlet.java
public class CheckoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws
IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        HttpSession s = req.getSession();
        Map<Integer, Integer> cart = (Map<Integer, Integer>) s.getAttribute("cart");
        double tax =
Double.parseDouble(getServletContext().getInitParameter("taxRate"));
        double sub = 0;
        double[] p = {0, 799, 399, 8999};
        if (cart != null) for (Map.Entry<Integer, Integer> e : cart.entrySet())
            sub += p[e.getKey()] * e.getValue();
        out.println("<h3>Bill</h3>Subtotal: " + sub + "<br>Tax (" + tax + "%): "
            + sub * tax / 100 + "<br>Total: " + (sub + sub * tax / 100));
        s.invalidate();
    }
}
// FILE: WEB-INF/web.xml (relevant part)
<context-param><param-name>taxRate</param-name><param-value>18</param-value></context-
param>
<servlet><servlet-name>Shop</servlet-name><servlet-class>ShopServlet</servlet-class>
  <init-param><param-name>shopName</param-name><param-value>TechMart</param-value></
init-param>
  <load-on-startup>1</load-on-startup></servlet>
<servlet-mapping><servlet-name>Shop</servlet-name><url-pattern>/shop</url-pattern></
servlet-mapping>
```

Output:

/shop lists the products of TechMart with Add to cart links and the cart item count. /checkout shows the bill with subtotal, 18% tax and total, and then ends the session so that the next visit starts with an empty cart.

Result:

The program was executed successfully and the expected output was obtained. The experiment on a complete servlet application with session management and configurable parameters has been implemented and verified.

Conclusion:

Thus, a complete servlet application with session management and configurable parameters was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 31

Develop a JSP-based Student Management System that performs CRUD operations using JavaBeans, JDBC, and MySQL while separating business logic from presentation logic.

Aim:

To develop a JSP-based Student Management System that performs CRUD operations using JavaBeans, JDBC and MySQL, keeping business logic separate from presentation logic.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

JavaServer Pages (JSP) allow Java code to be embedded in HTML and are compiled into servlets by the container. A JavaBean is a reusable class with a no-argument constructor, private properties and public getters and setters. To separate concerns, the business and database logic is placed in the bean and a DAO class (Data Access Object), while the JSP only displays data and collects input using <jsp:useBean>, <jsp:setProperty> and <jsp:getProperty>. JDBC with MySQL stores the data.

Procedure / Algorithm:

1. Create database college and table student(id, name, course, marks).
2. Create Student bean and StudentDAO with add(), update(), delete() and getAll() using PreparedStatement.
3. Create students.jsp with a form and a table; the action is processed with <jsp:useBean> and <jsp:setProperty>.
4. Call the DAO methods from the JSP based on the action parameter and display the list.
5. Deploy on Tomcat with the MySQL connector in WEB-INF/lib and test all operations.

Code / Program:

```
// FILE: Student.java  (JavaBean)
package model;
public class Student {
    private int id, marks; private String name, course;
    public int getId() { return id; }          public void setId(int v) { id = v; }
    public String getName() { return name; }   public void setName(String v) { name =
v; }
    public String getCourse() { return course; } public void setCourse(String v)
{ course = v; }
    public int getMarks() { return marks; }    public void setMarks(int v) { marks =
v; }
}
// FILE: StudentDAO.java  (business logic)
package model;
import java.sql.*; import java.util.*;
public class StudentDAO {
    static Connection con() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/college",
"root", "root");
    }
    public static void add(Student s) throws Exception {
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("INSERT
INTO student VALUES(?,?,?,?)")) {
            p.setInt(1, s.getId()); p.setString(2, s.getName()); p.setString(3,
s.getCourse());
            p.setInt(4, s.getMarks()); p.executeUpdate();
        }
    }
    public static void update(int id, int marks) throws Exception {
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("UPDATE
student SET marks=? WHERE id=?")) {
```

```
            p.setInt(1, marks); p.setInt(2, id); p.executeUpdate();
        }
    }
    public static void delete(int id) throws Exception {
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("DELETE
FROM student WHERE id=?")) {
            p.setInt(1, id); p.executeUpdate();
        }
    }
    public static List<Student> getAll() throws Exception {
        List<Student> l = new ArrayList<>();
        try (Connection c = con(); ResultSet r =
c.createStatement().executeQuery("SELECT * FROM student")) {
            while (r.next()) { Student s = new Student(); s.setId(r.getInt(1));
s.setName(r.getString(2));
                s.setCourse(r.getString(3)); s.setMarks(r.getInt(4)); l.add(s); }
        }
        return l;
    }
}
// FILE: students.jsp  (presentation)
<%@ page import="model.*" %>
<jsp:useBean id="st" class="model.Student" />
<jsp:setProperty name="st" property="*" />
<% String a = request.getParameter("action");
   if ("add".equals(a)) StudentDAO.add(st);
   else if ("update".equals(a)) StudentDAO.update(st.getId(), st.getMarks());
   else if ("delete".equals(a)) StudentDAO.delete(st.getId()); %>
<form method="post">ID <input name="id"> Name <input name="name"> Course <input
name="course">
  Marks <input name="marks"> <select
name="action"><option>add</option><option>update</option>
  <option>delete</option></select> <input type="submit"></form>
<table border="1"><tr><th>ID</th><th>Name</th><th>Course</th><th>Marks</th></tr>
<% for (Student s : StudentDAO.getAll()) { %>
<tr><td><%= s.getId() %></td><td><%= s.getName() %></td><td><%= s.getCourse()
%></td><td><%= s.getMarks() %></td></tr>
<% } %></table>
```

Output:

students.jsp displays a form and a table of all students. Adding, updating marks or deleting a student through the form immediately refreshes the table with the changed data.

Result:

The program was executed successfully and the expected output was obtained. The experiment on JavaBeans, JDBC and separation of business logic from presentation in JSP has been implemented and verified.

Conclusion:

Thus, JavaBeans, JDBC and separation of business logic from presentation in JSP was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 32

Create a secure JSP login application that authenticates users using a database, manages sessions, and handles invalid login attempts with custom error pages.

Aim:

To create a secure JSP login application that authenticates users from a database, manages sessions and handles invalid login attempts with custom error pages.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

A JSP login page posts credentials to a processing page that checks them against a users table using a PreparedStatement, which also protects against SQL injection. On success the user name is stored in the session implicit object; on failure the failed-attempt count is stored and the user is forwarded to a custom error page. A JSP can declare errorPage="error.jsp" in the page directive and the error page sets isErrorPage="true" to use the exception object. Secured pages check the session and redirect if it is missing.

Procedure / Algorithm:

1. Create table users(username, password) and insert sample rows.
2. Create login.jsp with a form that posts to auth.jsp.
3. In auth.jsp query the database with PreparedStatement and set session attributes on success.
4. Count invalid attempts in the session and forward to loginError.jsp; lock after 3 attempts.
5. Create dashboard.jsp that checks the session, logout.jsp that invalidates it, and error.jsp as the exception page.

Code / Program:

```
// FILE: login.jsp
<form action="auth.jsp" method="post">
  Username <input name="username"><br>
  Password <input type="password" name="password"><br>
  <input type="submit" value="Login">
</form>
<% if (request.getParameter("err") != null) { %>
  <p style="color:black">Invalid username or password.</p>
<% } %>
// FILE: auth.jsp
<%@ page import="java.sql.*" errorPage="error.jsp" %>
<%
  Class.forName("com.mysql.cj.jdbc.Driver");
  Integer tries = (Integer) session.getAttribute("tries");
  if (tries == null) tries = 0;
  if (tries >= 3) { response.sendRedirect("locked.jsp"); return; }
  try (Connection c = DriverManager.getConnection(
          "jdbc:mysql://localhost:3306/appdb", "root", "root");
       PreparedStatement p = c.prepareStatement(
          "SELECT * FROM users WHERE username=? AND password=?")) {
    p.setString(1, request.getParameter("username"));
    p.setString(2, request.getParameter("password"));
    ResultSet r = p.executeQuery();
    if (r.next()) {
      session.setAttribute("user", r.getString("username"));
      session.setAttribute("tries", 0);
      response.sendRedirect("dashboard.jsp");
    } else {
      session.setAttribute("tries", tries + 1);
      response.sendRedirect("login.jsp?err=1");
    }
  }
%>
// FILE: dashboard.jsp
```

```
<% if (session.getAttribute("user") == null) { response.sendRedirect("login.jsp");
return; } %>
<h3>Welcome, <%= session.getAttribute("user") %></h3>
<a href="logout.jsp">Logout</a>
// FILE: logout.jsp
<% session.invalidate(); response.sendRedirect("login.jsp"); %>
// FILE: error.jsp
<%@ page isErrorPage="true" %>
<h3>Something went wrong</h3>
<p>Details: <%= exception.getMessage() %></p>
<a href="login.jsp">Back to login</a>
```

Output:

Valid credentials open dashboard.jsp with the welcome message. An invalid login returns to the login page with an error message, three failed attempts redirect to a locked page, and database or runtime errors are shown on the custom error page.

Result:

The program was executed successfully and the expected output was obtained. The experiment on database authentication, sessions and custom error pages in JSP has been implemented and verified.

Conclusion:

Thus, database authentication, sessions and custom error pages in JSP was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 33

Design an online banking portal using JSP, JavaBeans, and JDBC to perform account management, fund transfers, and transaction history with proper exception handling.

Aim:

To design an online banking portal using JSP, JavaBeans and JDBC for account management, fund transfers and transaction history with proper exception handling.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

The portal keeps account data in an Account bean and uses a BankDAO class for database work. A fund transfer must be atomic, so it runs in a JDBC transaction: auto-commit is turned off, the debit, credit and history rows are executed and then commit() is called, with rollback() in the catch block if anything fails. Custom exceptions such as InsufficientFundsException are caught by the JSP and shown as messages. A transaction history table records every transfer with date, accounts and amount.

Procedure / Algorithm:

1. Create tables account(acc_no, name, balance) and txn(id, from_acc, to_acc, amount, txn_time).
2. Create the Account bean and BankDAO with getAccount(), transfer() and history().
3. In transfer() disable auto-commit, debit, credit, insert the history row, then commit.
4. Throw InsufficientFundsException when the balance is too low and roll back.
5. Create bank.jsp for balance, transfer form and history table; show exception messages.

Code / Program:

```
// FILE: BankDAO.java
package bank;
import java.sql.*; import java.util.*;
public class BankDAO {
    static Connection con() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/bankdb",
"root", "root");
    }
    public static double balance(int acc) throws Exception {
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("SELECT
balance FROM account WHERE acc_no=?")) {
            p.setInt(1, acc); ResultSet r = p.executeQuery();
            if (!r.next()) throw new Exception("Account not found");
            return r.getDouble(1);
        }
    }
    public static void transfer(int from, int to, double amt) throws Exception {
        try (Connection c = con()) {
            c.setAutoCommit(false);                        // transaction
            try {
                if (balance(from) < amt) throw new Exception("Insufficient balance");
                PreparedStatement d = c.prepareStatement("UPDATE account SET
balance=balance-? WHERE acc_no=?");
                d.setDouble(1, amt); d.setInt(2, from); d.executeUpdate();
                PreparedStatement e = c.prepareStatement("UPDATE account SET
balance=balance+? WHERE acc_no=?");
                e.setDouble(1, amt); e.setInt(2, to);
                if (e.executeUpdate() == 0) throw new Exception("Receiver not found");
                PreparedStatement h = c.prepareStatement("INSERT INTO
txn(from_acc,to_acc,amount,txn_time) VALUES(?,?,?,NOW())");
                h.setInt(1, from); h.setInt(2, to); h.setDouble(3, amt);
h.executeUpdate();
                c.commit();
            } catch (Exception ex) { c.rollback(); throw ex; }
        }
```

```
    }
    public static List<String> history(int acc) throws Exception {
        List<String> l = new ArrayList<>();
        try (Connection c = con(); PreparedStatement p = c.prepareStatement(
             "SELECT * FROM txn WHERE from_acc=? OR to_acc=? ORDER BY id DESC")) {
            p.setInt(1, acc); p.setInt(2, acc); ResultSet r = p.executeQuery();
            while (r.next()) l.add(r.getString("txn_time") + ": " +
r.getInt("from_acc") + " -> "
                + r.getInt("to_acc") + " Rs. " + r.getDouble("amount"));
        }
        return l;
    }
}
// FILE: bank.jsp
<%@ page import="bank.BankDAO" %>
<% int acc = 101; String msg = "";
   if (request.getParameter("to") != null) {
     try { BankDAO.transfer(acc, Integer.parseInt(request.getParameter("to")),
                            Double.parseDouble(request.getParameter("amt")));
           msg = "Transfer successful"; }
     catch (Exception e) { msg = "Failed: " + e.getMessage(); }
   } %>
<h3>Account <%= acc %> - Balance: Rs. <%= BankDAO.balance(acc) %></h3><p><%= msg
%></p>
<form method="post">To account <input name="to"> Amount <input name="amt"> <input
type="submit" value="Transfer"></form>
<h4>Transaction History</h4>
<% for (String h : BankDAO.history(acc)) { %><%= h %><br><% } %>
```

Output:

bank.jsp shows the balance of account 101. A valid transfer updates both balances and adds a row to the history list; an amount above the balance or a wrong receiver shows an error message and leaves the balances unchanged.

Result:

The program was executed successfully and the expected output was obtained. The experiment on transactional fund transfer and transaction history using JSP, beans and JDBC has been implemented and verified.

Conclusion:

Thus, transactional fund transfer and transaction history using JSP, beans and JDBC was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 34

Develop a JSP application demonstrating the complete JSP lifecycle, directives, scripting elements, standard actions, and implicit objects in a single project.

Aim:

To develop a JSP application that demonstrates the complete JSP life cycle, directives, scripting elements, standard actions and implicit objects in a single project.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

The JSP life cycle consists of translation to a servlet, compilation, loading, jspInit(), _jspService() for every request and jspDestroy(). Directives (page, include, taglib) give instructions to the container. Scripting elements are declarations <%! %>, scriptlets <% %>, expressions <%= %> and comments. Standard actions such as <jsp:include>, <jsp:forward>, <jsp:useBean>, <jsp:setProperty> and <jsp:getProperty> perform common tasks. The implicit objects are request, response, out, session, application, config, pageContext, page and exception.

Procedure / Algorithm:

1. Create lifecycle.jsp with a page directive, an include directive and a declaration containing jspInit() and jspDestroy().
2. Add scriptlets and expressions to compute and print values.
3. Use <jsp:useBean>, <jsp:setProperty> and <jsp:getProperty> with a simple bean.
4. Use <jsp:include> for header.jsp and show output from the implicit objects.
5. Deploy, open the page repeatedly and watch the console for life cycle messages.

Code / Program:

```
// FILE: lifecycle.jsp
<%@ page language="java" import="java.util.*" session="true" %>
<%@ include file="footer.html" %>
<%!
  int hits = 0;                                    // declaration
  public void jspInit()    { System.out.println("jspInit() called"); }
  public void jspDestroy() { System.out.println("jspDestroy() called"); }
  int square(int x) { return x * x; }
%>
<jsp:useBean id="user" class="beans.User" scope="session" />
<jsp:setProperty name="user" property="name" value="Rohit" />
<%-- JSP comment: not sent to the client --%>
<jsp:include page="header.jsp" />
<%
  hits++;                                           // scriptlet (_jspService)
  out.println("<h3>JSP Life Cycle Demo</h3>");
%>
<p>Hits since init: <%= hits %> | 7 squared = <%= square(7) %></p>
<p>User bean: <jsp:getProperty name="user" property="name" /></p>
<h4>Implicit objects</h4>
<ul>
  <li>request   : <%= request.getMethod() %> from <%= request.getRemoteAddr() %></li>
  <li>response  : content type <%= response.getContentType() %></li>
  <li>session   : id <%= session.getId() %></li>
  <li>application: <%= application.getServerInfo() %></li>
  <li>config    : servlet name <%= config.getServletName() %></li>
  <li>pageContext: <%= pageContext.getAttribute("x") %> (no attribute)</li>
  <li>page      : <%= page.getClass().getName() %></li>
</ul>
// FILE: beans/User.java
package beans;
public class User {
    private String name = "guest";
```

```
    public String getName() { return name; }
    public void setName(String n) { name = n; }
}
```

Output:

The page displays the included header and footer, the hit counter (increasing on every refresh), the square of 7, the bean property and the values of the implicit objects. The Tomcat console prints jspInit() on the first request and jspDestroy() when the application stops.

Result:

The program was executed successfully and the expected output was obtained. The experiment on the JSP life cycle, directives, scripting elements, actions and implicit objects has been implemented and verified.

Conclusion:

Thus, the JSP life cycle, directives, scripting elements, actions and implicit objects was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 35

Create a database-driven Inventory Management System using JSP and JDBC that supports Statement, PreparedStatement, and CallableStatement, and compares their usage.

Aim:

To create a database-driven Inventory Management System using JSP and JDBC that uses Statement, PreparedStatement and CallableStatement and compares their usage.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

Statement executes static SQL and is suitable for simple queries but is open to SQL injection when input is concatenated. PreparedStatement is precompiled, accepts ? parameters, is faster for repeated execution and is safe against injection. CallableStatement calls stored procedures with the escape syntax {call proc(?, ?)} and supports IN and OUT parameters, keeping business logic inside the database.

Procedure / Algorithm:

1. Create table item(id, name, qty, price) and the stored procedure restock(IN itemId, IN add, OUT newQty).
2. In inventory.jsp list all items with Statement.
3. Add a new item with PreparedStatement and parameters.
4. Call restock() with CallableStatement and read the OUT parameter.
5. Display a comparison table of the three statement types.

Code / Program:

```
-- Stored procedure (MySQL)
DELIMITER //
CREATE PROCEDURE restock(IN itemId INT, IN addQty INT, OUT newQty INT)
BEGIN
  UPDATE item SET qty = qty + addQty WHERE id = itemId;
  SELECT qty INTO newQty FROM item WHERE id = itemId;
END //
DELIMITER ;

// FILE: inventory.jsp
<%@ page import="java.sql.*" %>
<%
  Class.forName("com.mysql.cj.jdbc.Driver");
  Connection con = DriverManager.getConnection(
      "jdbc:mysql://localhost:3306/store", "root", "root");
  String msg = "";
  if ("add".equals(request.getParameter("action"))) {            // PreparedStatement
    PreparedStatement ps = con.prepareStatement("INSERT INTO item VALUES(?,?,?,?)");
    ps.setInt(1, Integer.parseInt(request.getParameter("id")));
    ps.setString(2, request.getParameter("name"));
    ps.setInt(3, Integer.parseInt(request.getParameter("qty")));
    ps.setDouble(4, Double.parseDouble(request.getParameter("price")));
    ps.executeUpdate(); msg = "Item added";
  }
  if ("restock".equals(request.getParameter("action"))) {        // CallableStatement
    CallableStatement cs = con.prepareCall("{call restock(?,?,?)}");
    cs.setInt(1, Integer.parseInt(request.getParameter("id")));
    cs.setInt(2, Integer.parseInt(request.getParameter("qty")));
    cs.registerOutParameter(3, Types.INTEGER);
    cs.execute(); msg = "New quantity = " + cs.getInt(3);
  }
%>
<h3>Inventory</h3><p><%= msg %></p>
<table border="1"><tr><th>ID</th><th>Name</th><th>Qty</th><th>Price</th></tr>
```

```
<%
  Statement st = con.createStatement();                          // Statement
  ResultSet rs = st.executeQuery("SELECT * FROM item");
  while (rs.next()) {
%>
<tr><td><%= rs.getInt(1) %></td><td><%= rs.getString(2) %></td>
    <td><%= rs.getInt(3) %></td><td><%= rs.getDouble(4) %></td></tr>
<% } con.close(); %>
</table>
<form method="post">ID <input name="id"> Name <input name="name"> Qty <input
name="qty">
  Price <input name="price"> <input type="hidden" name="action" value="add">
  <input type="submit" value="Add item"></form>
<form method="post">ID <input name="id"> Add qty <input name="qty">
  <input type="hidden" name="action" value="restock"><input type="submit"
value="Restock"></form>
<table border="1"><tr><th>Type</th><th>SQL</th><th>Speed</th><th>Safety</th></tr>
<tr><td>Statement</td><td>Static</td><td>Slower for repeats</td><td>Injection
risk</td></tr>
<tr><td>PreparedStatement</td><td>Parameterised</td><td>Precompiled,
fast</td><td>Safe</td></tr>
<tr><td>CallableStatement</td><td>Stored procedure</td><td>Fast, server
side</td><td>Safe, IN/OUT</td></tr></table>
```

Output:

The page lists the inventory with Statement, adds an item through PreparedStatement ("Item added"), shows "New quantity = n" after a restock through CallableStatement, and ends with the comparison table of the three statement types.

Result:

The program was executed successfully and the expected output was obtained. The experiment on Statement, PreparedStatement and CallableStatement in JSP has been implemented and verified.

Conclusion:

Thus, Statement, PreparedStatement and CallableStatement in JSP was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 36

Implement a JSP-based Employee Management System that connects to Oracle, MySQL, or MS SQL Server and performs CRUD operations using JavaBeans and MVC architecture.

Aim:

To implement a JSP-based Employee Management System that can connect to Oracle, MySQL or MS SQL Server and performs CRUD operations using JavaBeans and MVC architecture.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

MVC (Model-View-Controller) divides an application into the Model (Employee bean and EmployeeDAO), the View (JSP pages that only display data) and the Controller (a servlet that receives requests, calls the model and forwards to a view). Database independence is achieved by reading the driver class and URL from a properties file: Oracle uses oracle.jdbc.OracleDriver with jdbc:oracle:thin:@localhost:1521:xe, MySQL uses com.mysql.cj.jdbc.Driver with jdbc:mysql://localhost:3306/db, and SQL Server uses com.microsoft.sqlserver.jdbc.SQLServerDriver with jdbc:sqlserver://localhost:1433;databaseName=db.

Procedure / Algorithm:

1. Create table employee(id, name, dept, salary) in the chosen database.
2. Create the Employee bean and EmployeeDAO that gets driver and URL from db.properties.
3. Create EmployeeController servlet that handles add, edit, delete and list actions.
4. The controller stores the result as a request attribute and forwards to list.jsp or form.jsp.
5. Change only db.properties to switch between Oracle, MySQL and SQL Server and test.

Code / Program:

```
// FILE: db.properties
driver=com.mysql.cj.jdbc.Driver
url=jdbc:mysql://localhost:3306/company
user=root
password=root
# Oracle: oracle.jdbc.OracleDriver, jdbc:oracle:thin:@localhost:1521:xe
# SQL Server: com.microsoft.sqlserver.jdbc.SQLServerDriver,
jdbc:sqlserver://localhost:1433;databaseName=company
// FILE: Employee.java  (Model - bean)
package mvc;
public class Employee {
    private int id; private String name, dept; private double salary;
    public int getId() { return id; }            public void setId(int v) { id = v; }
    public String getName() { return name; }     public void setName(String v) { name
= v; }
    public String getDept() { return dept; }     public void setDept(String v) { dept
= v; }
    public double getSalary() { return salary; } public void setSalary(double v)
{ salary = v; }
}
// FILE: EmployeeDAO.java  (Model - database)
package mvc;
import java.sql.*; import java.util.*;
public class EmployeeDAO {
    Connection con() throws Exception {
        Properties p = new Properties();
        p.load(getClass().getResourceAsStream("/db.properties"));
        Class.forName(p.getProperty("driver"));
        return DriverManager.getConnection(p.getProperty("url"),
p.getProperty("user"), p.getProperty("password"));
    }
    void run(String sql, Object... v) throws Exception {          // insert / update /
delete
```

```
        try (Connection c = con(); PreparedStatement p = c.prepareStatement(sql)) {
            for (int i = 0; i < v.length; i++) p.setObject(i + 1, v[i]);
            p.executeUpdate();
        }
    }
    List<Employee> findAll() throws Exception {
        List<Employee> l = new ArrayList<>();
        try (Connection c = con(); ResultSet r =
c.createStatement().executeQuery("SELECT * FROM employee")) {
            while (r.next()) { Employee e = new Employee(); e.setId(r.getInt(1));
e.setName(r.getString(2));
                e.setDept(r.getString(3)); e.setSalary(r.getDouble(4)); l.add(e); }
        }
        return l;
    }
}
// FILE: EmployeeController.java  (Controller)
package mvc;
import java.io.IOException; import javax.servlet.*; import
javax.servlet.annotation.WebServlet; import javax.servlet.http.*;
@WebServlet("/employee")
public class EmployeeController extends HttpServlet {
    EmployeeDAO dao = new EmployeeDAO();
    protected void service(HttpServletRequest rq, HttpServletResponse rs) throws
ServletException, IOException {
        String a = rq.getParameter("action"), id = rq.getParameter("id"), sal =
rq.getParameter("salary");
        try {
            if ("add".equals(a)) dao.run("INSERT INTO employee VALUES(?,?,?,?)",
Integer.parseInt(id),
                    rq.getParameter("name"), rq.getParameter("dept"),
Double.parseDouble(sal));
            else if ("update".equals(a)) dao.run("UPDATE employee SET salary=? WHERE
id=?", Double.parseDouble(sal), Integer.parseInt(id));
            else if ("delete".equals(a)) dao.run("DELETE FROM employee WHERE id=?",
Integer.parseInt(id));
            rq.setAttribute("list", dao.findAll());
        } catch (Exception e) { rq.setAttribute("error", e.getMessage()); }
        rq.getRequestDispatcher("list.jsp").forward(rq, rs);
    }
}
// FILE: list.jsp  (View)
<%@ page import="java.util.*,mvc.Employee" %>
<p><%= request.getAttribute("error") == null ? "" : request.getAttribute("error")
%></p>
<table border="1"><tr><th>ID</th><th>Name</th><th>Dept</th><th>Salary</th></tr>
<% List<Employee> l = (List<Employee>) request.getAttribute("list");
   if (l != null) for (Employee e : l) { %>
<tr><td><%= e.getId() %></td><td><%= e.getName() %></td><td><%= e.getDept()
%></td><td><%= e.getSalary() %></td></tr>
<% } %></table>
<form action="employee" method="post">ID <input name="id"> Name <input name="name">
Dept <input name="dept">
  Salary <input name="salary"> <select
name="action"><option>add</option><option>update</option>
  <option>delete</option></select> <input type="submit"></form>
```

Output:

Requests to /employee are handled by the controller, which performs the CRUD operation and forwards to list.jsp; the employee table is displayed with the latest data. Changing only db.properties connects the same application to Oracle or SQL Server.

Result:

The program was executed successfully and the expected output was obtained. The experiment on MVC architecture and database-independent CRUD with JSP has been implemented and verified.

Conclusion:

Thus, MVC architecture and database-independent CRUD with JSP was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 37

Develop a JSP application that uses session management to implement a multi-step online examination system with automatic session timeout and recovery.

Aim:

To develop a JSP application that uses session management to implement a multi-step online examination with automatic session timeout and recovery.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

A multi-step exam presents one question per page and keeps the answers in the HttpSession so that they are not lost between steps. The session is given a limited life with session.setMaxInactiveInterval() or <session-timeout> in web.xml, after which the container invalidates it. For recovery, the last answered step and the answers are also saved in cookies (or the database), so after a timeout the candidate can resume from the last completed question instead of restarting.

Procedure / Algorithm:

1. Create start.jsp that reads the candidate name, creates the session and sets a 120-second timeout.
2. Create exam.jsp that shows question number q (from the parameter) and saves the previous answer in the session.
3. Save the last completed step in a cookie named lastStep after every question.
4. If the session is new or expired, redirect to start.jsp which offers recovery from the cookie.
5. Show the result page after the last question and invalidate the session.

Code / Program:

```
// FILE: start.jsp
<%
  Cookie[] cs = request.getCookies(); String last = null;
  if (cs != null) for (Cookie c : cs) if (c.getName().equals("lastStep")) last =
c.getValue();
  if (request.getParameter("name") != null) {
    session.setAttribute("name", request.getParameter("name"));
    session.setMaxInactiveInterval(120);                    // timeout
    String go = request.getParameter("resume") != null && last != null ? last : "1";
    response.sendRedirect("exam.jsp?q=" + go);
    return;
  }
%>
<form method="post">Name <input name="name"><br>
<% if (last != null) { %><input type="checkbox" name="resume"> Resume from question
<%= last %><br><% } %>
<input type="submit" value="Start exam"></form>
// FILE: exam.jsp
<%@ page import="java.util.*" %>
<%
  if (session.getAttribute("name") == null) { response.sendRedirect("start.jsp");
return; }
  String[] q = {"Which tag includes a file at translation time?",
                "Which object stores data for the whole application?",
                "Which method is called once when a JSP is initialised?"};
  String[][] o = {{"include directive", "jsp:forward"}, {"session", "application"},
                  {"jspInit()", "service()"}};
  String[] key = {"0", "1", "0"};
  int n = Integer.parseInt(request.getParameter("q"));
  String ans = request.getParameter("ans");
  if (ans != null) {
    session.setAttribute("a" + (n - 1), ans);
    Cookie c = new Cookie("lastStep", String.valueOf(n)); c.setMaxAge(3600);
    response.addCookie(c);
  }
```

```
  if (n > q.length) {
    int score = 0;
    for (int i = 0; i < q.length; i++) if (key[i].equals(session.getAttribute("a" +
i))) score++;
    out.println("<h3>Result for " + session.getAttribute("name") + ": " + score + "/"
+ q.length + "</h3>");
    session.invalidate(); return;
  }
%>
<h3>Question <%= n %> of <%= q.length %></h3><p><%= q[n - 1] %></p>
<form><input type="hidden" name="q" value="<%= n + 1 %>">
<% for (int i = 0; i < o[n - 1].length; i++) { %>
  <input type="radio" name="ans" value="<%= i %>"> <%= o[n - 1][i] %><br>
<% } %><input type="submit" value="Next"></form>
```

Output:

Each page shows one question; after the last question the result is displayed. If the session expires after 120 seconds of inactivity, the candidate is redirected to the start page, which offers to resume from the last saved question.

Result:

The program was executed successfully and the expected output was obtained. The experiment on session management, timeout and recovery in a multi-step JSP application has been implemented and verified.

Conclusion:

Thus, session management, timeout and recovery in a multi-step JSP application was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 38

Create a dynamic reporting system using JSP and JDBC that executes parameterized database queries and displays formatted reports with robust exception handling.

Aim:

To create a dynamic reporting system using JSP and JDBC that executes parameterized queries and displays formatted reports with robust exception handling.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

A dynamic report is generated from user-selected criteria such as department and minimum salary. The criteria are passed as parameters to a PreparedStatement, which prevents SQL injection and allows reuse of the compiled query. The ResultSet is formatted into an HTML table with totals. Robust exception handling combines input validation, try-catch-finally (or try-with-resources) for SQLException and NumberFormatException, and an errorPage so the user always sees a friendly message.

Procedure / Algorithm:

1. Create table employee(id, name, dept, salary) with sample data.
2. Create report.jsp with a form for department and minimum salary.
3. Validate the input and run SELECT ... WHERE dept=? AND salary>=? with PreparedStatement.
4. Format the rows in a table, compute the count and total salary and show them in the footer.
5. Catch NumberFormatException and SQLException and print messages; configure errorPage for other errors.

Code / Program:

```
// FILE: report.jsp
<%@ page import="java.sql.*,java.text.DecimalFormat" errorPage="reportError.jsp" %>
<form method="get">
  Department <select
name="dept"><option>IT</option><option>HR</option><option>Sales</option></select>
  Min salary <input name="min" value="0"> <input type="submit" value="Generate
report">
</form>
<%
  String dept = request.getParameter("dept");
  if (dept != null) {
    DecimalFormat f = new DecimalFormat("#,##0.00");
    try {
      double min = Double.parseDouble(request.getParameter("min"));
      Class.forName("com.mysql.cj.jdbc.Driver");
      try (Connection c = DriverManager.getConnection(
               "jdbc:mysql://localhost:3306/company", "root", "root");
           PreparedStatement p = c.prepareStatement(
               "SELECT id,name,salary FROM employee WHERE dept=? AND salary>=? ORDER
BY salary DESC")) {
        p.setString(1, dept); p.setDouble(2, min);
        ResultSet r = p.executeQuery();
        int count = 0; double total = 0;
%>
<h3><%= dept %> Report (salary >= <%= f.format(min) %>)</h3>
<table border="1"><tr><th>ID</th><th>Name</th><th>Salary</th></tr>
<%      while (r.next()) { count++; total += r.getDouble(3); %>
<tr><td><%= r.getInt(1) %></td><td><%= r.getString(2) %></td>
    <td align="right"><%= f.format(r.getDouble(3)) %></td></tr>
<%      } %>
<tr><td colspan="2"><b>Total (<%= count %> employees)</b></td>
    <td align="right"><b><%= f.format(total) %></b></td></tr></table>
<%    }
    } catch (NumberFormatException e) {
      out.println("<p>Please enter a valid number for salary.</p>");
```

```
    } catch (SQLException e) {
      out.println("<p>Database error: " + e.getMessage() + "</p>");
    }
  }
%>
// FILE: reportError.jsp
<%@ page isErrorPage="true" %>
<h3>Report could not be generated</h3><p><%= exception.getMessage() %></p>
```

Output:

Choosing a department and minimum salary displays a table of matching employees sorted by salary with the count and formatted total in the last row. Invalid salary input or database problems display clear messages instead of a stack trace.

Result:

The program was executed successfully and the expected output was obtained. The experiment on parameterised queries, formatted reports and exception handling has been implemented and verified.

Conclusion:

Thus, parameterised queries, formatted reports and exception handling was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 39

Design a JSP-based Library Management System using JavaBeans, JDBC, and MVC architecture, supporting book issue/return operations and database transaction management.

Aim:

To design a JSP-based Library Management System using JavaBeans, JDBC and MVC architecture that supports book issue and return operations with transaction management.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

In the MVC design the Book bean and LibraryDAO form the model, library.jsp is the view and LibraryController is the controller. Issuing a book changes two tables, so it is performed in one JDBC transaction: reduce the available copies in book, insert a row in issue_record and commit; if any step fails, rollback() restores the earlier state. Returning a book sets the return date, increases the available copies and calculates a fine for late returns, also in one transaction.

Procedure / Algorithm:

1. Create tables book(id, title, author, copies) and issue_record(id, book_id, member, issue_date, return_date).
2. Create the Book bean and LibraryDAO with issueBook(), returnBook() and listBooks().
3. In issueBook() disable auto-commit, check copies > 0, update, insert the record and commit.
4. Create LibraryController servlet that handles issue and return actions and forwards to library.jsp.
5. Display the book list with Issue and Return forms and test rollback with zero copies.

Code / Program:

```
// FILE: LibraryDAO.java
package lib;
import java.sql.*; import java.util.*;
public class LibraryDAO {
    Connection con() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/librarydb",
"root", "root");
    }
    public void issueBook(int bookId, String member) throws Exception {
        try (Connection c = con()) {
            c.setAutoCommit(false);
            try {
                PreparedStatement u = c.prepareStatement("UPDATE book SET
copies=copies-1 WHERE id=? AND copies>0");
                u.setInt(1, bookId);
                if (u.executeUpdate() == 0) throw new Exception("No copies
available");
                PreparedStatement i = c.prepareStatement("INSERT INTO
issue_record(book_id,member,issue_date) VALUES(?,?,CURDATE())");
                i.setInt(1, bookId); i.setString(2, member); i.executeUpdate();
                c.commit();
            } catch (Exception e) { c.rollback(); throw e; }
        }
    }
    public void returnBook(int rec) throws Exception {
        try (Connection c = con()) {
            c.setAutoCommit(false);
            try {
                PreparedStatement g = c.prepareStatement("SELECT book_id FROM
issue_record WHERE id=? AND return_date IS NULL");
                g.setInt(1, rec); ResultSet r = g.executeQuery();
```

```
                if (!r.next()) throw new Exception("Invalid or already returned
record");
                int book = r.getInt(1);
                PreparedStatement u = c.prepareStatement("UPDATE issue_record SET
return_date=CURDATE() WHERE id=?");
                u.setInt(1, rec); u.executeUpdate();
                PreparedStatement b = c.prepareStatement("UPDATE book SET
copies=copies+1 WHERE id=?");
                b.setInt(1, book); b.executeUpdate();
                c.commit();
            } catch (Exception e) { c.rollback(); throw e; }
        }
    }
    public List<String[]> listBooks() throws Exception {
        List<String[]> l = new ArrayList<>();
        try (Connection c = con(); ResultSet r =
c.createStatement().executeQuery("SELECT id,title,author,copies FROM book")) {
            while (r.next()) l.add(new String[]{r.getString(1), r.getString(2),
r.getString(3), r.getString(4)});
        }
        return l;
    }
}
// FILE: LibraryController.java
package lib;
import java.io.IOException; import javax.servlet.*; import
javax.servlet.annotation.WebServlet; import javax.servlet.http.*;
@WebServlet("/library")
public class LibraryController extends HttpServlet {
    LibraryDAO dao = new LibraryDAO();
    protected void service(HttpServletRequest rq, HttpServletResponse rs) throws
ServletException, IOException {
        try {
            String a = rq.getParameter("action");
            if ("issue".equals(a))
dao.issueBook(Integer.parseInt(rq.getParameter("id")), rq.getParameter("member"));
            else if ("return".equals(a))
dao.returnBook(Integer.parseInt(rq.getParameter("rec")));
            rq.setAttribute("books", dao.listBooks());
        } catch (Exception e) { rq.setAttribute("msg", e.getMessage()); }
        rq.getRequestDispatcher("library.jsp").forward(rq, rs);
    }
}
// FILE: library.jsp
<%@ page import="java.util.*" %>
<p><%= request.getAttribute("msg") == null ? "" : request.getAttribute("msg") %></p>
<table border="1"><tr><th>ID</th><th>Title</th><th>Author</th><th>Copies</th></tr>
<% List<String[]> b = (List<String[]>) request.getAttribute("books");
   if (b != null) for (String[] x : b) { %>
<tr><td><%= x[0] %></td><td><%= x[1] %></td><td><%= x[2] %></td><td><%= x[3]
%></td></tr><% } %></table>
<form action="library" method="post"><input type="hidden" name="action" value="issue">
  Book ID <input name="id"> Member <input name="member"><input type="submit"
value="Issue"></form>
<form action="library" method="post"><input type="hidden" name="action"
value="return">
  Record ID <input name="rec"><input type="submit" value="Return"></form>
```

Output:

The book list shows the available copies. Issuing a book reduces the copies and creates an issue record; returning it restores the copy. Trying to issue a book with zero copies rolls back the transaction and shows "No copies available".

Result:

The program was executed successfully and the expected output was obtained. The experiment on MVC design and transaction management for issue/return operations has been implemented and verified.

Conclusion:

Thus, MVC design and transaction management for issue/return operations was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 40

Develop a complete JSP-based E-Commerce application integrating JavaBeans, session management, JDBC, MVC architecture, database CRUD operations, and comprehensive error and exception handling.

Aim:

To develop a complete JSP-based E-Commerce application integrating JavaBeans, session management, JDBC, MVC architecture, database CRUD operations and comprehensive error and exception handling.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

The application combines the earlier concepts: Product and Cart beans (model), ProductDAO with JDBC for CRUD on products and orders, a front controller servlet (controller) and JSP pages for catalogue, cart and order summary (view). The Cart bean is kept in the session so each customer has an independent cart. Orders are saved in one transaction. Errors are handled at several levels: validation in the controller, try-catch for SQLException, and error-page entries in web.xml for HTTP 404/500 and java.lang.Exception.

Procedure / Algorithm:

1. Create tables product(id, name, price, stock) and orders(id, customer, total, order_date).
2. Create Product and Cart beans and ProductDAO with list(), add(), update(), delete() and saveOrder().
3. Create ShopController that handles view, add-to-cart, remove, update and checkout actions.
4. Store the Cart bean in the session; forward to catalogue.jsp, cart.jsp or invoice.jsp.
5. Register error pages in web.xml and test valid, invalid and failing operations.

Code / Program:

```
// FILE: Cart.java  (bean kept in session)
package shop;
import java.util.*;
public class Cart {
    private Map<Integer, Integer> items = new LinkedHashMap<>();
    public void add(int id) { items.merge(id, 1, Integer::sum); }
    public void remove(int id) { items.remove(id); }
    public Map<Integer, Integer> getItems() { return items; }
}
// FILE: ProductDAO.java
package shop;
import java.sql.*; import java.util.*;
public class ProductDAO {
    Connection con() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/shopdb",
"root", "root");
    }
    public List<Object[]> list() throws Exception {            // Read
        List<Object[]> l = new ArrayList<>();
        try (Connection c = con(); ResultSet r =
c.createStatement().executeQuery("SELECT id,name,price,stock FROM product")) {
            while (r.next()) l.add(new Object[]{r.getInt(1), r.getString(2),
r.getDouble(3), r.getInt(4)});
        }
        return l;
    }
    public void addProduct(String n, double price, int stock) throws Exception {   //
Create
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("INSERT
INTO product(name,price,stock) VALUES(?,?,?)")) {
            p.setString(1, n); p.setDouble(2, price); p.setInt(3, stock);
p.executeUpdate();
        }
```

```
    }
    public void deleteProduct(int id) throws Exception {                                //
Delete
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("DELETE
FROM product WHERE id=?")) {
            p.setInt(1, id); p.executeUpdate();
        }
    }
    public double checkout(String cust, Cart cart) throws Exception {                //
Update (stock) in a transaction
        try (Connection c = con()) {
            c.setAutoCommit(false);
            try {
                double total = 0;
                for (Map.Entry<Integer, Integer> e : cart.getItems().entrySet()) {
                    PreparedStatement g = c.prepareStatement("UPDATE product SET
stock=stock-? WHERE id=? AND stock>=?");
                    g.setInt(1, e.getValue()); g.setInt(2, e.getKey()); g.setInt(3,
e.getValue());
                    if (g.executeUpdate() == 0) throw new Exception("Out of stock:
product " + e.getKey());
                    PreparedStatement p = c.prepareStatement("SELECT price FROM
product WHERE id=?");
                    p.setInt(1, e.getKey()); ResultSet r = p.executeQuery(); r.next();
                    total += r.getDouble(1) * e.getValue();
                }
                PreparedStatement o = c.prepareStatement("INSERT INTO
orders(customer,total,order_date) VALUES(?,?,NOW())");
                o.setString(1, cust); o.setDouble(2, total); o.executeUpdate();
                c.commit(); return total;
            } catch (Exception e) { c.rollback(); throw e; }
        }
    }
}
// FILE: ShopController.java
package shop;
import java.io.IOException; import javax.servlet.*; import
javax.servlet.annotation.WebServlet; import javax.servlet.http.*;
@WebServlet("/shop")
public class ShopController extends HttpServlet {
    ProductDAO dao = new ProductDAO();
    protected void service(HttpServletRequest rq, HttpServletResponse rs) throws
ServletException, IOException {
        HttpSession s = rq.getSession();
        Cart cart = (Cart) s.getAttribute("cart");
        if (cart == null) { cart = new Cart(); s.setAttribute("cart", cart); }
        String a = rq.getParameter("action"), view = "catalogue.jsp";
        try {
            if ("add".equals(a)) cart.add(Integer.parseInt(rq.getParameter("id")));
            else if ("remove".equals(a))
cart.remove(Integer.parseInt(rq.getParameter("id")));
            else if ("checkout".equals(a)) {
                if (cart.getItems().isEmpty()) throw new Exception("Cart is empty");
                rq.setAttribute("total", dao.checkout(rq.getParameter("customer"),
cart));
                s.removeAttribute("cart"); view = "invoice.jsp";
            }
            rq.setAttribute("products", dao.list());
        } catch (Exception e) { rq.setAttribute("msg", e.getMessage()); }
        rq.getRequestDispatcher(view).forward(rq, rs);
    }
}
// FILE: catalogue.jsp
<%@ page import="java.util.*" errorPage="error.jsp" %>
<p><%= request.getAttribute("msg") == null ? "" : request.getAttribute("msg") %></p>
<% List<Object[]> p = (List<Object[]>) request.getAttribute("products");
   if (p != null) for (Object[] x : p) { %>
<%= x[1] %> - Rs. <%= x[2] %> (stock <%= x[3] %>) <a href="shop?action=add&id=<%= x[0]
%>">Add to cart</a><br>
<% } %>
<form action="shop"><input type="hidden" name="action" value="checkout">
  Name <input name="customer"> <input type="submit" value="Checkout"></form>
```

```
// FILE: invoice.jsp
<h3>Order placed. Total: Rs. <%= request.getAttribute("total") %></h3><a
href="shop">Continue shopping</a>
// FILE: error.jsp
<%@ page isErrorPage="true" %>
<h3>Error: <%= exception.getMessage() %></h3>
```

Output:

The catalogue lists products with Add to cart links; the cart is maintained in the session and checkout shows an invoice with the total and reduces the stock. Out-of-stock items, empty carts, invalid ids and missing pages show proper messages or error pages.

Result:

The program was executed successfully and the expected output was obtained. The experiment on a complete MVC e-commerce application with sessions, JDBC and error handling has been implemented and verified.

Conclusion:

Thus, a complete MVC e-commerce application with sessions, JDBC and error handling was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 41

Develop a JSP-based Student Management System that performs CRUD operations using JavaBeans, JDBC, and MySQL while separating business logic from presentation logic.

Aim:

To develop a JSP-based Student Management System that performs CRUD operations using JavaBeans, JDBC and MySQL, keeping business logic separate from presentation logic.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

JavaServer Pages (JSP) allow Java code to be embedded in HTML and are compiled into servlets by the container. A JavaBean is a reusable class with a no-argument constructor, private properties and public getters and setters. To separate concerns, the business and database logic is placed in the bean and a DAO class (Data Access Object), while the JSP only displays data and collects input using <jsp:useBean>, <jsp:setProperty> and <jsp:getProperty>. JDBC with MySQL stores the data.

Procedure / Algorithm:

1. Create database college and table student(id, name, course, marks).
2. Create Student bean and StudentDAO with add(), update(), delete() and getAll() using PreparedStatement.
3. Create students.jsp with a form and a table; the action is processed with <jsp:useBean> and <jsp:setProperty>.
4. Call the DAO methods from the JSP based on the action parameter and display the list.
5. Deploy on Tomcat with the MySQL connector in WEB-INF/lib and test all operations.

Code / Program:

```
// FILE: Student.java  (JavaBean)
package model;
public class Student {
    private int id, marks; private String name, course;
    public int getId() { return id; }          public void setId(int v) { id = v; }
    public String getName() { return name; }   public void setName(String v) { name =
v; }
    public String getCourse() { return course; } public void setCourse(String v)
{ course = v; }
    public int getMarks() { return marks; }    public void setMarks(int v) { marks =
v; }
}
// FILE: StudentDAO.java  (business logic)
package model;
import java.sql.*; import java.util.*;
public class StudentDAO {
    static Connection con() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/college",
"root", "root");
    }
    public static void add(Student s) throws Exception {
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("INSERT
INTO student VALUES(?,?,?,?)")) {
            p.setInt(1, s.getId()); p.setString(2, s.getName()); p.setString(3,
s.getCourse());
            p.setInt(4, s.getMarks()); p.executeUpdate();
        }
    }
    public static void update(int id, int marks) throws Exception {
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("UPDATE
student SET marks=? WHERE id=?")) {
```

```
            p.setInt(1, marks); p.setInt(2, id); p.executeUpdate();
        }
    }
    public static void delete(int id) throws Exception {
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("DELETE
FROM student WHERE id=?")) {
            p.setInt(1, id); p.executeUpdate();
        }
    }
    public static List<Student> getAll() throws Exception {
        List<Student> l = new ArrayList<>();
        try (Connection c = con(); ResultSet r =
c.createStatement().executeQuery("SELECT * FROM student")) {
            while (r.next()) { Student s = new Student(); s.setId(r.getInt(1));
s.setName(r.getString(2));
                s.setCourse(r.getString(3)); s.setMarks(r.getInt(4)); l.add(s); }
        }
        return l;
    }
}
// FILE: students.jsp  (presentation)
<%@ page import="model.*" %>
<jsp:useBean id="st" class="model.Student" />
<jsp:setProperty name="st" property="*" />
<% String a = request.getParameter("action");
   if ("add".equals(a)) StudentDAO.add(st);
   else if ("update".equals(a)) StudentDAO.update(st.getId(), st.getMarks());
   else if ("delete".equals(a)) StudentDAO.delete(st.getId()); %>
<form method="post">ID <input name="id"> Name <input name="name"> Course <input
name="course">
  Marks <input name="marks"> <select
name="action"><option>add</option><option>update</option>
  <option>delete</option></select> <input type="submit"></form>
<table border="1"><tr><th>ID</th><th>Name</th><th>Course</th><th>Marks</th></tr>
<% for (Student s : StudentDAO.getAll()) { %>
<tr><td><%= s.getId() %></td><td><%= s.getName() %></td><td><%= s.getCourse()
%></td><td><%= s.getMarks() %></td></tr>
<% } %></table>
```

Output:

students.jsp displays a form and a table of all students. Adding, updating marks or deleting a student through the form immediately refreshes the table with the changed data.

Result:

The program was executed successfully and the expected output was obtained. The experiment on JavaBeans, JDBC and separation of business logic from presentation in JSP has been implemented and verified.

Conclusion:

Thus, JavaBeans, JDBC and separation of business logic from presentation in JSP was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 42

Create a secure JSP login application that authenticates users using a database, manages sessions, and handles invalid login attempts with custom error pages.

Aim:

To create a secure JSP login application that authenticates users from a database, manages sessions and handles invalid login attempts with custom error pages.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

A JSP login page posts credentials to a processing page that checks them against a users table using a PreparedStatement, which also protects against SQL injection. On success the user name is stored in the session implicit object; on failure the failed-attempt count is stored and the user is forwarded to a custom error page. A JSP can declare errorPage="error.jsp" in the page directive and the error page sets isErrorPage="true" to use the exception object. Secured pages check the session and redirect if it is missing.

Procedure / Algorithm:

1. Create table users(username, password) and insert sample rows.
2. Create login.jsp with a form that posts to auth.jsp.
3. In auth.jsp query the database with PreparedStatement and set session attributes on success.
4. Count invalid attempts in the session and forward to loginError.jsp; lock after 3 attempts.
5. Create dashboard.jsp that checks the session, logout.jsp that invalidates it, and error.jsp as the exception page.

Code / Program:

```
// FILE: login.jsp
<form action="auth.jsp" method="post">
  Username <input name="username"><br>
  Password <input type="password" name="password"><br>
  <input type="submit" value="Login">
</form>
<% if (request.getParameter("err") != null) { %>
  <p style="color:black">Invalid username or password.</p>
<% } %>
// FILE: auth.jsp
<%@ page import="java.sql.*" errorPage="error.jsp" %>
<%
  Class.forName("com.mysql.cj.jdbc.Driver");
  Integer tries = (Integer) session.getAttribute("tries");
  if (tries == null) tries = 0;
  if (tries >= 3) { response.sendRedirect("locked.jsp"); return; }
  try (Connection c = DriverManager.getConnection(
          "jdbc:mysql://localhost:3306/appdb", "root", "root");
       PreparedStatement p = c.prepareStatement(
          "SELECT * FROM users WHERE username=? AND password=?")) {
    p.setString(1, request.getParameter("username"));
    p.setString(2, request.getParameter("password"));
    ResultSet r = p.executeQuery();
    if (r.next()) {
      session.setAttribute("user", r.getString("username"));
      session.setAttribute("tries", 0);
      response.sendRedirect("dashboard.jsp");
    } else {
      session.setAttribute("tries", tries + 1);
      response.sendRedirect("login.jsp?err=1");
    }
  }
%>
// FILE: dashboard.jsp
```

```
<% if (session.getAttribute("user") == null) { response.sendRedirect("login.jsp");
return; } %>
<h3>Welcome, <%= session.getAttribute("user") %></h3>
<a href="logout.jsp">Logout</a>
// FILE: logout.jsp
<% session.invalidate(); response.sendRedirect("login.jsp"); %>
// FILE: error.jsp
<%@ page isErrorPage="true" %>
<h3>Something went wrong</h3>
<p>Details: <%= exception.getMessage() %></p>
<a href="login.jsp">Back to login</a>
```

Output:

Valid credentials open dashboard.jsp with the welcome message. An invalid login returns to the login page with an error message, three failed attempts redirect to a locked page, and database or runtime errors are shown on the custom error page.

Result:

The program was executed successfully and the expected output was obtained. The experiment on database authentication, sessions and custom error pages in JSP has been implemented and verified.

Conclusion:

Thus, database authentication, sessions and custom error pages in JSP was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 43

Design an online banking portal using JSP, JavaBeans, and JDBC to perform account management, fund transfers, and transaction history with proper exception handling.

Aim:

To design an online banking portal using JSP, JavaBeans and JDBC for account management, fund transfers and transaction history with proper exception handling.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

The portal keeps account data in an Account bean and uses a BankDAO class for database work. A fund transfer must be atomic, so it runs in a JDBC transaction: auto-commit is turned off, the debit, credit and history rows are executed and then commit() is called, with rollback() in the catch block if anything fails. Custom exceptions such as InsufficientFundsException are caught by the JSP and shown as messages. A transaction history table records every transfer with date, accounts and amount.

Procedure / Algorithm:

1. Create tables account(acc_no, name, balance) and txn(id, from_acc, to_acc, amount, txn_time).
2. Create the Account bean and BankDAO with getAccount(), transfer() and history().
3. In transfer() disable auto-commit, debit, credit, insert the history row, then commit.
4. Throw InsufficientFundsException when the balance is too low and roll back.
5. Create bank.jsp for balance, transfer form and history table; show exception messages.

Code / Program:

```
// FILE: BankDAO.java
package bank;
import java.sql.*; import java.util.*;
public class BankDAO {
    static Connection con() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/bankdb",
"root", "root");
    }
    public static double balance(int acc) throws Exception {
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("SELECT
balance FROM account WHERE acc_no=?")) {
            p.setInt(1, acc); ResultSet r = p.executeQuery();
            if (!r.next()) throw new Exception("Account not found");
            return r.getDouble(1);
        }
    }
    public static void transfer(int from, int to, double amt) throws Exception {
        try (Connection c = con()) {
            c.setAutoCommit(false);                        // transaction
            try {
                if (balance(from) < amt) throw new Exception("Insufficient balance");
                PreparedStatement d = c.prepareStatement("UPDATE account SET
balance=balance-? WHERE acc_no=?");
                d.setDouble(1, amt); d.setInt(2, from); d.executeUpdate();
                PreparedStatement e = c.prepareStatement("UPDATE account SET
balance=balance+? WHERE acc_no=?");
                e.setDouble(1, amt); e.setInt(2, to);
                if (e.executeUpdate() == 0) throw new Exception("Receiver not found");
                PreparedStatement h = c.prepareStatement("INSERT INTO
txn(from_acc,to_acc,amount,txn_time) VALUES(?,?,?,NOW())");
                h.setInt(1, from); h.setInt(2, to); h.setDouble(3, amt);
h.executeUpdate();
                c.commit();
            } catch (Exception ex) { c.rollback(); throw ex; }
        }
```

```
    }
    public static List<String> history(int acc) throws Exception {
        List<String> l = new ArrayList<>();
        try (Connection c = con(); PreparedStatement p = c.prepareStatement(
             "SELECT * FROM txn WHERE from_acc=? OR to_acc=? ORDER BY id DESC")) {
            p.setInt(1, acc); p.setInt(2, acc); ResultSet r = p.executeQuery();
            while (r.next()) l.add(r.getString("txn_time") + ": " +
r.getInt("from_acc") + " -> "
                + r.getInt("to_acc") + " Rs. " + r.getDouble("amount"));
        }
        return l;
    }
}
// FILE: bank.jsp
<%@ page import="bank.BankDAO" %>
<% int acc = 101; String msg = "";
   if (request.getParameter("to") != null) {
     try { BankDAO.transfer(acc, Integer.parseInt(request.getParameter("to")),
                            Double.parseDouble(request.getParameter("amt")));
           msg = "Transfer successful"; }
     catch (Exception e) { msg = "Failed: " + e.getMessage(); }
   } %>
<h3>Account <%= acc %> - Balance: Rs. <%= BankDAO.balance(acc) %></h3><p><%= msg
%></p>
<form method="post">To account <input name="to"> Amount <input name="amt"> <input
type="submit" value="Transfer"></form>
<h4>Transaction History</h4>
<% for (String h : BankDAO.history(acc)) { %><%= h %><br><% } %>
```

Output:

bank.jsp shows the balance of account 101. A valid transfer updates both balances and adds a row to the history list; an amount above the balance or a wrong receiver shows an error message and leaves the balances unchanged.

Result:

The program was executed successfully and the expected output was obtained. The experiment on transactional fund transfer and transaction history using JSP, beans and JDBC has been implemented and verified.

Conclusion:

Thus, transactional fund transfer and transaction history using JSP, beans and JDBC was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 44

Develop a JSP application demonstrating the complete JSP lifecycle, directives, scripting elements, standard actions, and implicit objects in a single project.

Aim:

To develop a JSP application that demonstrates the complete JSP life cycle, directives, scripting elements, standard actions and implicit objects in a single project.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

The JSP life cycle consists of translation to a servlet, compilation, loading, jspInit(), _jspService() for every request and jspDestroy(). Directives (page, include, taglib) give instructions to the container. Scripting elements are declarations <%! %>, scriptlets <% %>, expressions <%= %> and comments. Standard actions such as <jsp:include>, <jsp:forward>, <jsp:useBean>, <jsp:setProperty> and <jsp:getProperty> perform common tasks. The implicit objects are request, response, out, session, application, config, pageContext, page and exception.

Procedure / Algorithm:

1. Create lifecycle.jsp with a page directive, an include directive and a declaration containing jspInit() and jspDestroy().
2. Add scriptlets and expressions to compute and print values.
3. Use <jsp:useBean>, <jsp:setProperty> and <jsp:getProperty> with a simple bean.
4. Use <jsp:include> for header.jsp and show output from the implicit objects.
5. Deploy, open the page repeatedly and watch the console for life cycle messages.

Code / Program:

```
// FILE: lifecycle.jsp
<%@ page language="java" import="java.util.*" session="true" %>
<%@ include file="footer.html" %>
<%!
  int hits = 0;                                    // declaration
  public void jspInit()    { System.out.println("jspInit() called"); }
  public void jspDestroy() { System.out.println("jspDestroy() called"); }
  int square(int x) { return x * x; }
%>
<jsp:useBean id="user" class="beans.User" scope="session" />
<jsp:setProperty name="user" property="name" value="Rohit" />
<%-- JSP comment: not sent to the client --%>
<jsp:include page="header.jsp" />
<%
  hits++;                                           // scriptlet (_jspService)
  out.println("<h3>JSP Life Cycle Demo</h3>");
%>
<p>Hits since init: <%= hits %> | 7 squared = <%= square(7) %></p>
<p>User bean: <jsp:getProperty name="user" property="name" /></p>
<h4>Implicit objects</h4>
<ul>
  <li>request   : <%= request.getMethod() %> from <%= request.getRemoteAddr() %></li>
  <li>response  : content type <%= response.getContentType() %></li>
  <li>session   : id <%= session.getId() %></li>
  <li>application: <%= application.getServerInfo() %></li>
  <li>config    : servlet name <%= config.getServletName() %></li>
  <li>pageContext: <%= pageContext.getAttribute("x") %> (no attribute)</li>
  <li>page      : <%= page.getClass().getName() %></li>
</ul>
// FILE: beans/User.java
package beans;
public class User {
    private String name = "guest";
```

```
    public String getName() { return name; }
    public void setName(String n) { name = n; }
}
```

Output:

The page displays the included header and footer, the hit counter (increasing on every refresh), the square of 7, the bean property and the values of the implicit objects. The Tomcat console prints jspInit() on the first request and jspDestroy() when the application stops.

Result:

The program was executed successfully and the expected output was obtained. The experiment on the JSP life cycle, directives, scripting elements, actions and implicit objects has been implemented and verified.

Conclusion:

Thus, the JSP life cycle, directives, scripting elements, actions and implicit objects was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 45

Create a database-driven Inventory Management System using JSP and JDBC that supports Statement, PreparedStatement, and CallableStatement, and compares their usage.

Aim:

To create a database-driven Inventory Management System using JSP and JDBC that uses Statement, PreparedStatement and CallableStatement and compares their usage.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

Statement executes static SQL and is suitable for simple queries but is open to SQL injection when input is concatenated. PreparedStatement is precompiled, accepts ? parameters, is faster for repeated execution and is safe against injection. CallableStatement calls stored procedures with the escape syntax {call proc(?, ?)} and supports IN and OUT parameters, keeping business logic inside the database.

Procedure / Algorithm:

1. Create table item(id, name, qty, price) and the stored procedure restock(IN itemId, IN add, OUT newQty).
2. In inventory.jsp list all items with Statement.
3. Add a new item with PreparedStatement and parameters.
4. Call restock() with CallableStatement and read the OUT parameter.
5. Display a comparison table of the three statement types.

Code / Program:

```
-- Stored procedure (MySQL)
DELIMITER //
CREATE PROCEDURE restock(IN itemId INT, IN addQty INT, OUT newQty INT)
BEGIN
  UPDATE item SET qty = qty + addQty WHERE id = itemId;
  SELECT qty INTO newQty FROM item WHERE id = itemId;
END //
DELIMITER ;

// FILE: inventory.jsp
<%@ page import="java.sql.*" %>
<%
  Class.forName("com.mysql.cj.jdbc.Driver");
  Connection con = DriverManager.getConnection(
      "jdbc:mysql://localhost:3306/store", "root", "root");
  String msg = "";
  if ("add".equals(request.getParameter("action"))) {            // PreparedStatement
    PreparedStatement ps = con.prepareStatement("INSERT INTO item VALUES(?,?,?,?)");
    ps.setInt(1, Integer.parseInt(request.getParameter("id")));
    ps.setString(2, request.getParameter("name"));
    ps.setInt(3, Integer.parseInt(request.getParameter("qty")));
    ps.setDouble(4, Double.parseDouble(request.getParameter("price")));
    ps.executeUpdate(); msg = "Item added";
  }
  if ("restock".equals(request.getParameter("action"))) {        // CallableStatement
    CallableStatement cs = con.prepareCall("{call restock(?,?,?)}");
    cs.setInt(1, Integer.parseInt(request.getParameter("id")));
    cs.setInt(2, Integer.parseInt(request.getParameter("qty")));
    cs.registerOutParameter(3, Types.INTEGER);
    cs.execute(); msg = "New quantity = " + cs.getInt(3);
  }
%>
<h3>Inventory</h3><p><%= msg %></p>
<table border="1"><tr><th>ID</th><th>Name</th><th>Qty</th><th>Price</th></tr>
```

```
<%
  Statement st = con.createStatement();                          // Statement
  ResultSet rs = st.executeQuery("SELECT * FROM item");
  while (rs.next()) {
%>
<tr><td><%= rs.getInt(1) %></td><td><%= rs.getString(2) %></td>
    <td><%= rs.getInt(3) %></td><td><%= rs.getDouble(4) %></td></tr>
<% } con.close(); %>
</table>
<form method="post">ID <input name="id"> Name <input name="name"> Qty <input
name="qty">
  Price <input name="price"> <input type="hidden" name="action" value="add">
  <input type="submit" value="Add item"></form>
<form method="post">ID <input name="id"> Add qty <input name="qty">
  <input type="hidden" name="action" value="restock"><input type="submit"
value="Restock"></form>
<table border="1"><tr><th>Type</th><th>SQL</th><th>Speed</th><th>Safety</th></tr>
<tr><td>Statement</td><td>Static</td><td>Slower for repeats</td><td>Injection
risk</td></tr>
<tr><td>PreparedStatement</td><td>Parameterised</td><td>Precompiled,
fast</td><td>Safe</td></tr>
<tr><td>CallableStatement</td><td>Stored procedure</td><td>Fast, server
side</td><td>Safe, IN/OUT</td></tr></table>
```

Output:

The page lists the inventory with Statement, adds an item through PreparedStatement ("Item added"), shows "New quantity = n" after a restock through CallableStatement, and ends with the comparison table of the three statement types.

Result:

The program was executed successfully and the expected output was obtained. The experiment on Statement, PreparedStatement and CallableStatement in JSP has been implemented and verified.

Conclusion:

Thus, Statement, PreparedStatement and CallableStatement in JSP was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 46

Implement a JSP-based Employee Management System that connects to Oracle, MySQL, or MS SQL Server and performs CRUD operations using JavaBeans and MVC architecture.

Aim:

To implement a JSP-based Employee Management System that can connect to Oracle, MySQL or MS SQL Server and performs CRUD operations using JavaBeans and MVC architecture.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

MVC (Model-View-Controller) divides an application into the Model (Employee bean and EmployeeDAO), the View (JSP pages that only display data) and the Controller (a servlet that receives requests, calls the model and forwards to a view). Database independence is achieved by reading the driver class and URL from a properties file: Oracle uses oracle.jdbc.OracleDriver with jdbc:oracle:thin:@localhost:1521:xe, MySQL uses com.mysql.cj.jdbc.Driver with jdbc:mysql://localhost:3306/db, and SQL Server uses com.microsoft.sqlserver.jdbc.SQLServerDriver with jdbc:sqlserver://localhost:1433;databaseName=db.

Procedure / Algorithm:

1. Create table employee(id, name, dept, salary) in the chosen database.
2. Create the Employee bean and EmployeeDAO that gets driver and URL from db.properties.
3. Create EmployeeController servlet that handles add, edit, delete and list actions.
4. The controller stores the result as a request attribute and forwards to list.jsp or form.jsp.
5. Change only db.properties to switch between Oracle, MySQL and SQL Server and test.

Code / Program:

```
// FILE: db.properties
driver=com.mysql.cj.jdbc.Driver
url=jdbc:mysql://localhost:3306/company
user=root
password=root
# Oracle: oracle.jdbc.OracleDriver, jdbc:oracle:thin:@localhost:1521:xe
# SQL Server: com.microsoft.sqlserver.jdbc.SQLServerDriver,
jdbc:sqlserver://localhost:1433;databaseName=company
// FILE: Employee.java  (Model - bean)
package mvc;
public class Employee {
    private int id; private String name, dept; private double salary;
    public int getId() { return id; }            public void setId(int v) { id = v; }
    public String getName() { return name; }     public void setName(String v) { name
= v; }
    public String getDept() { return dept; }     public void setDept(String v) { dept
= v; }
    public double getSalary() { return salary; } public void setSalary(double v)
{ salary = v; }
}
// FILE: EmployeeDAO.java  (Model - database)
package mvc;
import java.sql.*; import java.util.*;
public class EmployeeDAO {
    Connection con() throws Exception {
        Properties p = new Properties();
        p.load(getClass().getResourceAsStream("/db.properties"));
        Class.forName(p.getProperty("driver"));
        return DriverManager.getConnection(p.getProperty("url"),
p.getProperty("user"), p.getProperty("password"));
    }
    void run(String sql, Object... v) throws Exception {          // insert / update /
delete
```

```
        try (Connection c = con(); PreparedStatement p = c.prepareStatement(sql)) {
            for (int i = 0; i < v.length; i++) p.setObject(i + 1, v[i]);
            p.executeUpdate();
        }
    }
    List<Employee> findAll() throws Exception {
        List<Employee> l = new ArrayList<>();
        try (Connection c = con(); ResultSet r =
c.createStatement().executeQuery("SELECT * FROM employee")) {
            while (r.next()) { Employee e = new Employee(); e.setId(r.getInt(1));
e.setName(r.getString(2));
                e.setDept(r.getString(3)); e.setSalary(r.getDouble(4)); l.add(e); }
        }
        return l;
    }
}
// FILE: EmployeeController.java  (Controller)
package mvc;
import java.io.IOException; import javax.servlet.*; import
javax.servlet.annotation.WebServlet; import javax.servlet.http.*;
@WebServlet("/employee")
public class EmployeeController extends HttpServlet {
    EmployeeDAO dao = new EmployeeDAO();
    protected void service(HttpServletRequest rq, HttpServletResponse rs) throws
ServletException, IOException {
        String a = rq.getParameter("action"), id = rq.getParameter("id"), sal =
rq.getParameter("salary");
        try {
            if ("add".equals(a)) dao.run("INSERT INTO employee VALUES(?,?,?,?)",
Integer.parseInt(id),
                    rq.getParameter("name"), rq.getParameter("dept"),
Double.parseDouble(sal));
            else if ("update".equals(a)) dao.run("UPDATE employee SET salary=? WHERE
id=?", Double.parseDouble(sal), Integer.parseInt(id));
            else if ("delete".equals(a)) dao.run("DELETE FROM employee WHERE id=?",
Integer.parseInt(id));
            rq.setAttribute("list", dao.findAll());
        } catch (Exception e) { rq.setAttribute("error", e.getMessage()); }
        rq.getRequestDispatcher("list.jsp").forward(rq, rs);
    }
}
// FILE: list.jsp  (View)
<%@ page import="java.util.*,mvc.Employee" %>
<p><%= request.getAttribute("error") == null ? "" : request.getAttribute("error")
%></p>
<table border="1"><tr><th>ID</th><th>Name</th><th>Dept</th><th>Salary</th></tr>
<% List<Employee> l = (List<Employee>) request.getAttribute("list");
   if (l != null) for (Employee e : l) { %>
<tr><td><%= e.getId() %></td><td><%= e.getName() %></td><td><%= e.getDept()
%></td><td><%= e.getSalary() %></td></tr>
<% } %></table>
<form action="employee" method="post">ID <input name="id"> Name <input name="name">
Dept <input name="dept">
  Salary <input name="salary"> <select
name="action"><option>add</option><option>update</option>
  <option>delete</option></select> <input type="submit"></form>
```

Output:

Requests to /employee are handled by the controller, which performs the CRUD operation and forwards to list.jsp; the employee table is displayed with the latest data. Changing only db.properties connects the same application to Oracle or SQL Server.

Result:

The program was executed successfully and the expected output was obtained. The experiment on MVC architecture and database-independent CRUD with JSP has been implemented and verified.

Conclusion:

Thus, MVC architecture and database-independent CRUD with JSP was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 47

Develop a JSP application that uses session management to implement a multi-step online examination system with automatic session timeout and recovery.

Aim:

To develop a JSP application that uses session management to implement a multi-step online examination with automatic session timeout and recovery.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

A multi-step exam presents one question per page and keeps the answers in the HttpSession so that they are not lost between steps. The session is given a limited life with session.setMaxInactiveInterval() or <session-timeout> in web.xml, after which the container invalidates it. For recovery, the last answered step and the answers are also saved in cookies (or the database), so after a timeout the candidate can resume from the last completed question instead of restarting.

Procedure / Algorithm:

1. Create start.jsp that reads the candidate name, creates the session and sets a 120-second timeout.
2. Create exam.jsp that shows question number q (from the parameter) and saves the previous answer in the session.
3. Save the last completed step in a cookie named lastStep after every question.
4. If the session is new or expired, redirect to start.jsp which offers recovery from the cookie.
5. Show the result page after the last question and invalidate the session.

Code / Program:

```
// FILE: start.jsp
<%
  Cookie[] cs = request.getCookies(); String last = null;
  if (cs != null) for (Cookie c : cs) if (c.getName().equals("lastStep")) last =
c.getValue();
  if (request.getParameter("name") != null) {
    session.setAttribute("name", request.getParameter("name"));
    session.setMaxInactiveInterval(120);                    // timeout
    String go = request.getParameter("resume") != null && last != null ? last : "1";
    response.sendRedirect("exam.jsp?q=" + go);
    return;
  }
%>
<form method="post">Name <input name="name"><br>
<% if (last != null) { %><input type="checkbox" name="resume"> Resume from question
<%= last %><br><% } %>
<input type="submit" value="Start exam"></form>
// FILE: exam.jsp
<%@ page import="java.util.*" %>
<%
  if (session.getAttribute("name") == null) { response.sendRedirect("start.jsp");
return; }
  String[] q = {"Which tag includes a file at translation time?",
                "Which object stores data for the whole application?",
                "Which method is called once when a JSP is initialised?"};
  String[][] o = {{"include directive", "jsp:forward"}, {"session", "application"},
                  {"jspInit()", "service()"}};
  String[] key = {"0", "1", "0"};
  int n = Integer.parseInt(request.getParameter("q"));
  String ans = request.getParameter("ans");
  if (ans != null) {
    session.setAttribute("a" + (n - 1), ans);
    Cookie c = new Cookie("lastStep", String.valueOf(n)); c.setMaxAge(3600);
    response.addCookie(c);
  }
```

```
  if (n > q.length) {
    int score = 0;
    for (int i = 0; i < q.length; i++) if (key[i].equals(session.getAttribute("a" +
i))) score++;
    out.println("<h3>Result for " + session.getAttribute("name") + ": " + score + "/"
+ q.length + "</h3>");
    session.invalidate(); return;
  }
%>
<h3>Question <%= n %> of <%= q.length %></h3><p><%= q[n - 1] %></p>
<form><input type="hidden" name="q" value="<%= n + 1 %>">
<% for (int i = 0; i < o[n - 1].length; i++) { %>
  <input type="radio" name="ans" value="<%= i %>"> <%= o[n - 1][i] %><br>
<% } %><input type="submit" value="Next"></form>
```

Output:

Each page shows one question; after the last question the result is displayed. If the session expires after 120 seconds of inactivity, the candidate is redirected to the start page, which offers to resume from the last saved question.

Result:

The program was executed successfully and the expected output was obtained. The experiment on session management, timeout and recovery in a multi-step JSP application has been implemented and verified.

Conclusion:

Thus, session management, timeout and recovery in a multi-step JSP application was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 48

Create a dynamic reporting system using JSP and JDBC that executes parameterized database queries and displays formatted reports with robust exception handling.

Aim:

To create a dynamic reporting system using JSP and JDBC that executes parameterized queries and displays formatted reports with robust exception handling.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

A dynamic report is generated from user-selected criteria such as department and minimum salary. The criteria are passed as parameters to a PreparedStatement, which prevents SQL injection and allows reuse of the compiled query. The ResultSet is formatted into an HTML table with totals. Robust exception handling combines input validation, try-catch-finally (or try-with-resources) for SQLException and NumberFormatException, and an errorPage so the user always sees a friendly message.

Procedure / Algorithm:

1. Create table employee(id, name, dept, salary) with sample data.
2. Create report.jsp with a form for department and minimum salary.
3. Validate the input and run SELECT ... WHERE dept=? AND salary>=? with PreparedStatement.
4. Format the rows in a table, compute the count and total salary and show them in the footer.
5. Catch NumberFormatException and SQLException and print messages; configure errorPage for other errors.

Code / Program:

```
// FILE: report.jsp
<%@ page import="java.sql.*,java.text.DecimalFormat" errorPage="reportError.jsp" %>
<form method="get">
  Department <select
name="dept"><option>IT</option><option>HR</option><option>Sales</option></select>
  Min salary <input name="min" value="0"> <input type="submit" value="Generate
report">
</form>
<%
  String dept = request.getParameter("dept");
  if (dept != null) {
    DecimalFormat f = new DecimalFormat("#,##0.00");
    try {
      double min = Double.parseDouble(request.getParameter("min"));
      Class.forName("com.mysql.cj.jdbc.Driver");
      try (Connection c = DriverManager.getConnection(
               "jdbc:mysql://localhost:3306/company", "root", "root");
           PreparedStatement p = c.prepareStatement(
               "SELECT id,name,salary FROM employee WHERE dept=? AND salary>=? ORDER
BY salary DESC")) {
        p.setString(1, dept); p.setDouble(2, min);
        ResultSet r = p.executeQuery();
        int count = 0; double total = 0;
%>
<h3><%= dept %> Report (salary >= <%= f.format(min) %>)</h3>
<table border="1"><tr><th>ID</th><th>Name</th><th>Salary</th></tr>
<%      while (r.next()) { count++; total += r.getDouble(3); %>
<tr><td><%= r.getInt(1) %></td><td><%= r.getString(2) %></td>
    <td align="right"><%= f.format(r.getDouble(3)) %></td></tr>
<%      } %>
<tr><td colspan="2"><b>Total (<%= count %> employees)</b></td>
    <td align="right"><b><%= f.format(total) %></b></td></tr></table>
<%    }
    } catch (NumberFormatException e) {
      out.println("<p>Please enter a valid number for salary.</p>");
```

```
    } catch (SQLException e) {
      out.println("<p>Database error: " + e.getMessage() + "</p>");
    }
  }
%>
// FILE: reportError.jsp
<%@ page isErrorPage="true" %>
<h3>Report could not be generated</h3><p><%= exception.getMessage() %></p>
```

Output:

Choosing a department and minimum salary displays a table of matching employees sorted by salary with the count and formatted total in the last row. Invalid salary input or database problems display clear messages instead of a stack trace.

Result:

The program was executed successfully and the expected output was obtained. The experiment on parameterised queries, formatted reports and exception handling has been implemented and verified.

Conclusion:

Thus, parameterised queries, formatted reports and exception handling was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 49

Design a JSP-based Library Management System using JavaBeans, JDBC, and MVC architecture, supporting book issue/return operations and database transaction management.

Aim:

To design a JSP-based Library Management System using JavaBeans, JDBC and MVC architecture that supports book issue and return operations with transaction management.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

In the MVC design the Book bean and LibraryDAO form the model, library.jsp is the view and LibraryController is the controller. Issuing a book changes two tables, so it is performed in one JDBC transaction: reduce the available copies in book, insert a row in issue_record and commit; if any step fails, rollback() restores the earlier state. Returning a book sets the return date, increases the available copies and calculates a fine for late returns, also in one transaction.

Procedure / Algorithm:

1. Create tables book(id, title, author, copies) and issue_record(id, book_id, member, issue_date, return_date).
2. Create the Book bean and LibraryDAO with issueBook(), returnBook() and listBooks().
3. In issueBook() disable auto-commit, check copies > 0, update, insert the record and commit.
4. Create LibraryController servlet that handles issue and return actions and forwards to library.jsp.
5. Display the book list with Issue and Return forms and test rollback with zero copies.

Code / Program:

```
// FILE: LibraryDAO.java
package lib;
import java.sql.*; import java.util.*;
public class LibraryDAO {
    Connection con() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/librarydb",
"root", "root");
    }
    public void issueBook(int bookId, String member) throws Exception {
        try (Connection c = con()) {
            c.setAutoCommit(false);
            try {
                PreparedStatement u = c.prepareStatement("UPDATE book SET
copies=copies-1 WHERE id=? AND copies>0");
                u.setInt(1, bookId);
                if (u.executeUpdate() == 0) throw new Exception("No copies
available");
                PreparedStatement i = c.prepareStatement("INSERT INTO
issue_record(book_id,member,issue_date) VALUES(?,?,CURDATE())");
                i.setInt(1, bookId); i.setString(2, member); i.executeUpdate();
                c.commit();
            } catch (Exception e) { c.rollback(); throw e; }
        }
    }
    public void returnBook(int rec) throws Exception {
        try (Connection c = con()) {
            c.setAutoCommit(false);
            try {
                PreparedStatement g = c.prepareStatement("SELECT book_id FROM
issue_record WHERE id=? AND return_date IS NULL");
                g.setInt(1, rec); ResultSet r = g.executeQuery();
```

```
                if (!r.next()) throw new Exception("Invalid or already returned
record");
                int book = r.getInt(1);
                PreparedStatement u = c.prepareStatement("UPDATE issue_record SET
return_date=CURDATE() WHERE id=?");
                u.setInt(1, rec); u.executeUpdate();
                PreparedStatement b = c.prepareStatement("UPDATE book SET
copies=copies+1 WHERE id=?");
                b.setInt(1, book); b.executeUpdate();
                c.commit();
            } catch (Exception e) { c.rollback(); throw e; }
        }
    }
    public List<String[]> listBooks() throws Exception {
        List<String[]> l = new ArrayList<>();
        try (Connection c = con(); ResultSet r =
c.createStatement().executeQuery("SELECT id,title,author,copies FROM book")) {
            while (r.next()) l.add(new String[]{r.getString(1), r.getString(2),
r.getString(3), r.getString(4)});
        }
        return l;
    }
}
// FILE: LibraryController.java
package lib;
import java.io.IOException; import javax.servlet.*; import
javax.servlet.annotation.WebServlet; import javax.servlet.http.*;
@WebServlet("/library")
public class LibraryController extends HttpServlet {
    LibraryDAO dao = new LibraryDAO();
    protected void service(HttpServletRequest rq, HttpServletResponse rs) throws
ServletException, IOException {
        try {
            String a = rq.getParameter("action");
            if ("issue".equals(a))
dao.issueBook(Integer.parseInt(rq.getParameter("id")), rq.getParameter("member"));
            else if ("return".equals(a))
dao.returnBook(Integer.parseInt(rq.getParameter("rec")));
            rq.setAttribute("books", dao.listBooks());
        } catch (Exception e) { rq.setAttribute("msg", e.getMessage()); }
        rq.getRequestDispatcher("library.jsp").forward(rq, rs);
    }
}
// FILE: library.jsp
<%@ page import="java.util.*" %>
<p><%= request.getAttribute("msg") == null ? "" : request.getAttribute("msg") %></p>
<table border="1"><tr><th>ID</th><th>Title</th><th>Author</th><th>Copies</th></tr>
<% List<String[]> b = (List<String[]>) request.getAttribute("books");
   if (b != null) for (String[] x : b) { %>
<tr><td><%= x[0] %></td><td><%= x[1] %></td><td><%= x[2] %></td><td><%= x[3]
%></td></tr><% } %></table>
<form action="library" method="post"><input type="hidden" name="action" value="issue">
  Book ID <input name="id"> Member <input name="member"><input type="submit"
value="Issue"></form>
<form action="library" method="post"><input type="hidden" name="action"
value="return">
  Record ID <input name="rec"><input type="submit" value="Return"></form>
```

Output:

The book list shows the available copies. Issuing a book reduces the copies and creates an issue record; returning it restores the copy. Trying to issue a book with zero copies rolls back the transaction and shows "No copies available".

Result:

The program was executed successfully and the expected output was obtained. The experiment on MVC design and transaction management for issue/return operations has been implemented and verified.

Conclusion:

Thus, MVC design and transaction management for issue/return operations was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 50

Develop a complete JSP-based E-Commerce application integrating JavaBeans, session management, JDBC, MVC architecture, database CRUD operations, and comprehensive error and exception handling.

Aim:

To develop a complete JSP-based E-Commerce application integrating JavaBeans, session management, JDBC, MVC architecture, database CRUD operations and comprehensive error and exception handling.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, MySQL Server 8.x, MySQL Connector/J, JSP / JSTL libraries, IDE (NetBeans / Eclipse), web browser; PC with minimum 4 GB RAM.

Theory:

The application combines the earlier concepts: Product and Cart beans (model), ProductDAO with JDBC for CRUD on products and orders, a front controller servlet (controller) and JSP pages for catalogue, cart and order summary (view). The Cart bean is kept in the session so each customer has an independent cart. Orders are saved in one transaction. Errors are handled at several levels: validation in the controller, try-catch for SQLException, and error-page entries in web.xml for HTTP 404/500 and java.lang.Exception.

Procedure / Algorithm:

1. Create tables product(id, name, price, stock) and orders(id, customer, total, order_date).
2. Create Product and Cart beans and ProductDAO with list(), add(), update(), delete() and saveOrder().
3. Create ShopController that handles view, add-to-cart, remove, update and checkout actions.
4. Store the Cart bean in the session; forward to catalogue.jsp, cart.jsp or invoice.jsp.
5. Register error pages in web.xml and test valid, invalid and failing operations.

Code / Program:

```
// FILE: Cart.java  (bean kept in session)
package shop;
import java.util.*;
public class Cart {
    private Map<Integer, Integer> items = new LinkedHashMap<>();
    public void add(int id) { items.merge(id, 1, Integer::sum); }
    public void remove(int id) { items.remove(id); }
    public Map<Integer, Integer> getItems() { return items; }
}
// FILE: ProductDAO.java
package shop;
import java.sql.*; import java.util.*;
public class ProductDAO {
    Connection con() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection("jdbc:mysql://localhost:3306/shopdb",
"root", "root");
    }
    public List<Object[]> list() throws Exception {            // Read
        List<Object[]> l = new ArrayList<>();
        try (Connection c = con(); ResultSet r =
c.createStatement().executeQuery("SELECT id,name,price,stock FROM product")) {
            while (r.next()) l.add(new Object[]{r.getInt(1), r.getString(2),
r.getDouble(3), r.getInt(4)});
        }
        return l;
    }
    public void addProduct(String n, double price, int stock) throws Exception {   //
Create
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("INSERT
INTO product(name,price,stock) VALUES(?,?,?)")) {
            p.setString(1, n); p.setDouble(2, price); p.setInt(3, stock);
p.executeUpdate();
        }
```

```
    }
    public void deleteProduct(int id) throws Exception {                                //
Delete
        try (Connection c = con(); PreparedStatement p = c.prepareStatement("DELETE
FROM product WHERE id=?")) {
            p.setInt(1, id); p.executeUpdate();
        }
    }
    public double checkout(String cust, Cart cart) throws Exception {                //
Update (stock) in a transaction
        try (Connection c = con()) {
            c.setAutoCommit(false);
            try {
                double total = 0;
                for (Map.Entry<Integer, Integer> e : cart.getItems().entrySet()) {
                    PreparedStatement g = c.prepareStatement("UPDATE product SET
stock=stock-? WHERE id=? AND stock>=?");
                    g.setInt(1, e.getValue()); g.setInt(2, e.getKey()); g.setInt(3,
e.getValue());
                    if (g.executeUpdate() == 0) throw new Exception("Out of stock:
product " + e.getKey());
                    PreparedStatement p = c.prepareStatement("SELECT price FROM
product WHERE id=?");
                    p.setInt(1, e.getKey()); ResultSet r = p.executeQuery(); r.next();
                    total += r.getDouble(1) * e.getValue();
                }
                PreparedStatement o = c.prepareStatement("INSERT INTO
orders(customer,total,order_date) VALUES(?,?,NOW())");
                o.setString(1, cust); o.setDouble(2, total); o.executeUpdate();
                c.commit(); return total;
            } catch (Exception e) { c.rollback(); throw e; }
        }
    }
}
// FILE: ShopController.java
package shop;
import java.io.IOException; import javax.servlet.*; import
javax.servlet.annotation.WebServlet; import javax.servlet.http.*;
@WebServlet("/shop")
public class ShopController extends HttpServlet {
    ProductDAO dao = new ProductDAO();
    protected void service(HttpServletRequest rq, HttpServletResponse rs) throws
ServletException, IOException {
        HttpSession s = rq.getSession();
        Cart cart = (Cart) s.getAttribute("cart");
        if (cart == null) { cart = new Cart(); s.setAttribute("cart", cart); }
        String a = rq.getParameter("action"), view = "catalogue.jsp";
        try {
            if ("add".equals(a)) cart.add(Integer.parseInt(rq.getParameter("id")));
            else if ("remove".equals(a))
cart.remove(Integer.parseInt(rq.getParameter("id")));
            else if ("checkout".equals(a)) {
                if (cart.getItems().isEmpty()) throw new Exception("Cart is empty");
                rq.setAttribute("total", dao.checkout(rq.getParameter("customer"),
cart));
                s.removeAttribute("cart"); view = "invoice.jsp";
            }
            rq.setAttribute("products", dao.list());
        } catch (Exception e) { rq.setAttribute("msg", e.getMessage()); }
        rq.getRequestDispatcher(view).forward(rq, rs);
    }
}
// FILE: catalogue.jsp
<%@ page import="java.util.*" errorPage="error.jsp" %>
<p><%= request.getAttribute("msg") == null ? "" : request.getAttribute("msg") %></p>
<% List<Object[]> p = (List<Object[]>) request.getAttribute("products");
   if (p != null) for (Object[] x : p) { %>
<%= x[1] %> - Rs. <%= x[2] %> (stock <%= x[3] %>) <a href="shop?action=add&id=<%= x[0]
%>">Add to cart</a><br>
<% } %>
<form action="shop"><input type="hidden" name="action" value="checkout">
  Name <input name="customer"> <input type="submit" value="Checkout"></form>
```

```
// FILE: invoice.jsp
<h3>Order placed. Total: Rs. <%= request.getAttribute("total") %></h3><a
href="shop">Continue shopping</a>
// FILE: error.jsp
<%@ page isErrorPage="true" %>
<h3>Error: <%= exception.getMessage() %></h3>
```

Output:

The catalogue lists products with Add to cart links; the cart is maintained in the session and checkout shows an invoice with the total and reduces the stock. Out-of-stock items, empty carts, invalid ids and missing pages show proper messages or error pages.

Result:

The program was executed successfully and the expected output was obtained. The experiment on a complete MVC e-commerce application with sessions, JDBC and error handling has been implemented and verified.

Conclusion:

Thus, a complete MVC e-commerce application with sessions, JDBC and error handling was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 51

Write a Java program to demonstrate the concept of inheritance using a parent class and a child class. Create methods in both classes and access the parent class properties and methods through the child class.

Aim:

To demonstrate inheritance using a parent class and a child class, and to access the properties and methods of the parent class through the child class.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Inheritance is an object-oriented mechanism in which a child (sub) class acquires the properties and methods of a parent (super) class using the extends keyword. It promotes code reuse and allows method overriding. Non-private members of the parent are directly accessible in the child class, the super keyword refers to the parent class members and constructor, and an object of the child class can call both its own and the inherited methods.

Procedure / Algorithm:

1. Create a parent class Vehicle with the properties brand and speed and the methods setData() and showVehicle().
2. Create a child class Car that extends Vehicle and adds the property doors and the method showCar().
3. In showCar() access the parent property brand directly and call the parent method using super.
4. In main() create an object of Car and call the parent and child methods through it.
5. Compile and run the program and observe the output.

Code / Program:

```
class Vehicle {
    String brand;
    int speed;
    void setData(String brand, int speed) {
        this.brand = brand;
        this.speed = speed;
    }
    void showVehicle() {
        System.out.println("Brand : " + brand);
        System.out.println("Speed : " + speed + " km/h");
    }
}

class Car extends Vehicle {
    int doors;
    void setDoors(int d) { doors = d; }
    void showCar() {
        System.out.println("--- Car Details ---");
        super.showVehicle();                        // parent method
        System.out.println("Brand (parent property) : " + brand);
        System.out.println("Doors : " + doors);
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Car c = new Car();
        c.setData("Toyota", 180);     // parent method through child object
        c.setDoors(4);
        c.showVehicle();              // inherited method
        c.showCar();                  // child method
    }
```

```
}
```

Output:

```
Brand : Toyota
Speed : 180 km/h
--- Car Details ---
Brand : Toyota
Speed : 180 km/h
Brand (parent property) : Toyota
Doors : 4
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on single inheritance and access to parent members through a child object has been implemented and verified.

Conclusion:

Thus, single inheritance and access to parent members through a child object was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 52

Write a Java program to perform basic arithmetic operations (Addition, Subtraction, Multiplication, Division and Modulus) on two numbers using user input.

Aim:

To perform the basic arithmetic operations (addition, subtraction, multiplication, division and modulus) on two numbers accepted from the user.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Java provides the arithmetic operators +, -, *, / and % for numeric data. The / operator performs integer division when both operands are integers, so double is used to obtain the fractional result, and % returns the remainder. User input is read from the keyboard with the Scanner class of java.util, using methods such as nextDouble(). Division and modulus by zero must be checked to avoid ArithmeticException (for integers) or Infinity/NaN (for floating-point values).

Procedure / Algorithm:

1. Import java.util.Scanner and create a Scanner object for System.in.
2. Read the two numbers a and b from the user.
3. Calculate the sum, difference, product, quotient and remainder.
4. Check whether b is zero before performing division and modulus.
5. Display all results.

Code / Program:

```
import java.util.Scanner;

public class Arithmetic {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number  : ");
        double a = sc.nextDouble();
        System.out.print("Enter second number : ");
        double b = sc.nextDouble();

        System.out.println("Addition       : " + (a + b));
        System.out.println("Subtraction    : " + (a - b));
        System.out.println("Multiplication : " + (a * b));
        if (b != 0) {
            System.out.println("Division       : " + (a / b));
            System.out.println("Modulus        : " + (a % b));
        } else {
            System.out.println("Division and modulus are not possible (second number
is 0)");
        }
        sc.close();
    }
}
```

Output:

Sample run (inputs 20 and 6): Enter first number : 20 Enter second number : 6 Addition : 26.0 Subtraction : 14.0 Multiplication : 120.0 Division : 3.3333333333333335 Modulus : 2.0

Result:

The program was executed successfully and the expected output was obtained. The experiment on arithmetic operators and user input with Scanner has been implemented and verified.

Conclusion:

Thus, arithmetic operators and user input with Scanner was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 53

Write a Java program to create a user-defined package, define a class inside the package, and access its methods from another Java class using the import statement.

Aim:

To create a user-defined package containing a class and to access its methods from another class using the import statement.

Software / Hardware Required:

JDK 17 or later, text editor / IDE (NetBeans, Eclipse or IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A package is a namespace that groups related classes and interfaces; it corresponds to a folder structure and is declared with the package statement as the first line of the source file. Classes and methods that must be used outside the package are declared public. Another class uses them by writing import packageName.ClassName; (or packageName.*;). The package is compiled with javac -d . File.java, which creates the folders, and the program is run using the fully qualified name of the main class if it is also in a package.

Procedure / Algorithm:

1. Create the file mypack/MathOps.java with the statement package mypack; and a public class MathOps with public methods add(), multiply() and isPrime().
2. Compile it using javac -d . mypack/MathOps.java.
3. Create the file PackageTest.java in the parent folder and import mypack.MathOps.
4. Create an object of MathOps and call its methods.
5. Compile with javac PackageTest.java and run with java PackageTest.

Code / Program:

```
// FILE: mypack/MathOps.java
package mypack;

public class MathOps {
    public int add(int a, int b) { return a + b; }
    public int multiply(int a, int b) { return a * b; }
    public boolean isPrime(int n) {
        if (n < 2) return false;
        for (int i = 2; i * i <= n; i++)
            if (n % i == 0) return false;
        return true;
    }
}
// FILE: PackageTest.java
import mypack.MathOps;

public class PackageTest {
    public static void main(String[] args) {
        MathOps m = new MathOps();
        System.out.println("Sum of 15 and 25      : " + m.add(15, 25));
        System.out.println("Product of 6 and 7    : " + m.multiply(6, 7));
        System.out.println("Is 29 a prime number? : " + m.isPrime(29));
        System.out.println("Is 30 a prime number? : " + m.isPrime(30));
    }
}
```

Output:

```
Sum of 15 and 25      : 40
Product of 6 and 7    : 42
Is 29 a prime number? : true
Is 30 a prime number? : false
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on user-defined packages and the import statement has been implemented and verified.

Conclusion:

Thus, user-defined packages and the import statement was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 54

Write a Java program to establish a database connection with MySQL using JDBC by loading the MySQL Connector/J driver. Perform the following CRUD operations on a Student table: Create – Insert a new student record; Read – Display all student records; Update – Update an existing student record; Delete – Delete a student record.

Aim:

To establish a connection with a MySQL database using JDBC by loading the MySQL Connector/J driver and to perform Create, Read, Update and Delete operations on a Student table.

Software / Hardware Required:

JDK 17 or later, MySQL Server 8.x, MySQL Connector/J (JDBC driver JAR), IDE (NetBeans / Eclipse / IntelliJ IDEA), Command Prompt / Terminal; PC with minimum 4 GB RAM.

Theory:

JDBC (Java Database Connectivity) is the Java API for accessing relational databases. MySQL Connector/J is the JDBC driver for MySQL; it is loaded with Class.forName("com.mysql.cj.jdbc.Driver") and a connection is obtained with DriverManager.getConnection(url, user, password). A PreparedStatement executes parameterised SQL: executeUpdate() is used for INSERT, UPDATE and DELETE statements and executeQuery() returns a ResultSet for SELECT statements. Resources are closed using try-with-resources.

Procedure / Algorithm:

1. Start MySQL and create the database and table: CREATE DATABASE college; CREATE TABLE student(id INT PRIMARY KEY, name VARCHAR(50), course VARCHAR(30), marks INT);
2. Add the mysql-connector-j JAR to the classpath.
3. Load the driver and open a connection using the URL jdbc:mysql://localhost:3306/college.
4. Write methods insert(), display(), update() and delete() using PreparedStatement.
5. Call the methods in sequence from main() and display the table after each step.

Code / Program:

```
import java.sql.*;

public class StudentCRUD {
    static final String URL = "jdbc:mysql://localhost:3306/college";
    static final String USER = "root", PASS = "root";

    static void insert(Connection c, int id, String name, String course, int marks)
throws SQLException {
        String sql = "INSERT INTO student(id, name, course, marks) VALUES(?,?,?,?)";
        try (PreparedStatement p = c.prepareStatement(sql)) {
            p.setInt(1, id); p.setString(2, name);
            p.setString(3, course); p.setInt(4, marks);
            System.out.println(p.executeUpdate() + " record inserted");
        }
    }
    static void display(Connection c) throws SQLException {
        try (Statement s = c.createStatement();
             ResultSet r = s.executeQuery("SELECT * FROM student")) {
            System.out.println("ID\tName\tCourse\tMarks");
            while (r.next())
                System.out.println(r.getInt(1) + "\t" + r.getString(2) + "\t"
                    + r.getString(3) + "\t" + r.getInt(4));
        }
    }
    static void update(Connection c, int id, int marks) throws SQLException {
        try (PreparedStatement p = c.prepareStatement("UPDATE student SET marks=?
WHERE id=?")) {
            p.setInt(1, marks); p.setInt(2, id);
            System.out.println(p.executeUpdate() + " record updated");
        }
    }
```

```
    static void delete(Connection c, int id) throws SQLException {
        try (PreparedStatement p = c.prepareStatement("DELETE FROM student WHERE
id=?")) {
            p.setInt(1, id);
            System.out.println(p.executeUpdate() + " record deleted");
        }
    }
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");          // load Connector/J
            try (Connection con = DriverManager.getConnection(URL, USER, PASS)) {
                System.out.println("Connected to MySQL");
                insert(con, 1, "Rohit", "M.Tech", 85);
                insert(con, 2, "Amit", "M.Tech", 78);
                display(con);
                update(con, 2, 82);
                display(con);
                delete(con, 1);
                display(con);
            }
        } catch (ClassNotFoundException e) {
            System.out.println("Driver not found: " + e.getMessage());
        } catch (SQLException e) {
            System.out.println("SQL error: " + e.getMessage());
        }
    }
}
```

Output:

```
Connected to MySQL
1 record inserted
1 record inserted
ID  Name   Course  Marks
1   Rohit  M.Tech  85
2   Amit   M.Tech  78
1 record updated  (marks of id 2 changed to 82)
1 record deleted  (id 1 removed)
ID  Name   Course  Marks
2   Amit   M.Tech  82
```

Result:

The program was executed successfully and the expected output was obtained. The experiment on JDBC connectivity and CRUD operations using MySQL Connector/J has been implemented and verified.

Conclusion:

Thus, JDBC connectivity and CRUD operations using MySQL Connector/J was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.

Experiment No. 55

Write a Java Servlet program using Apache Tomcat to accept user input through an HTML form and display the submitted data on a web page. Demonstrate the use of doGet() and doPost() methods.

Aim:

To write a Java Servlet that accepts user input through an HTML form and displays the submitted data on a web page, demonstrating the doGet() and doPost() methods on Apache Tomcat.

Software / Hardware Required:

JDK 17 or later, Apache Tomcat 9.x, Servlet API 4.0 (servlet-api.jar), IDE (NetBeans / Eclipse), web browser (Chrome / Firefox); PC with minimum 4 GB RAM.

Theory:

A servlet is a server-side Java class that extends HttpServlet and is executed by a servlet container such as Apache Tomcat. The container calls doGet() for HTTP GET requests, in which the form data is appended to the URL and is therefore visible and limited in length, and doPost() for HTTP POST requests, in which the data is sent in the request body and is suitable for sensitive or large data. Form values are read with request.getParameter(), and the response is written through response.getWriter() after setting the content type. The servlet is mapped to a URL with @WebServlet or in web.xml.

Procedure / Algorithm:

1. Create a dynamic web project and place form.html in the web content folder.
2. Create FormServlet extending HttpServlet and map it to /submit with @WebServlet.
3. Override doGet() to read the parameters and display them with the method name GET.
4. Override doPost() to read the parameters and display them with the method name POST.
5. Deploy on Tomcat, open http://localhost:8080/FormApp/form.html and submit the form using both methods.

Code / Program:

```
// FILE: form.html
<html>
<body>
  <h3>Student Form (GET)</h3>
  <form action="submit" method="get">
    Name: <input type="text" name="name"><br>
    Email: <input type="text" name="email"><br>
    Course: <input type="text" name="course"><br>
    <input type="submit" value="Submit using GET">
  </form>
  <h3>Student Form (POST)</h3>
  <form action="submit" method="post">
    Name: <input type="text" name="name"><br>
    Email: <input type="text" name="email"><br>
    Course: <input type="text" name="course"><br>
    <input type="submit" value="Submit using POST">
  </form>
</body>
</html>
// FILE: FormServlet.java
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/submit")
public class FormServlet extends HttpServlet {
    private void show(HttpServletRequest req, HttpServletResponse res, String method)
            throws IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
```

```
        out.println("<html><body>");
        out.println("<h3>Data received using " + method + "</h3>");
        out.println("Name   : " + req.getParameter("name") + "<br>");
        out.println("Email  : " + req.getParameter("email") + "<br>");
        out.println("Course : " + req.getParameter("course") + "<br>");
        out.println("</body></html>");
    }
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        show(req, res, "GET");
    }
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        show(req, res, "POST");
    }
}
```

Output:

The form page is displayed in the browser. After pressing Submit, the servlet page shows "Data received using GET" (with the values visible in the URL, e.g. /submit?name=Rohit&email=...) or "Data received using POST" (URL unchanged), followed by the submitted Name, Email and Course.

Result:

The program was executed successfully and the expected output was obtained. The experiment on servlets, HTML forms and the doGet() and doPost() methods has been implemented and verified.

Conclusion:

Thus, servlets, HTML forms and the doGet() and doPost() methods was studied and implemented in Java. The experiment gave practical understanding of the concept and its use in real applications.